
Bitcoin Utilities Documentation

Release 0.8.3

Konstantinos Karasavvas

May 05, 2026

USER GUIDE

1	Security Model	3
2	Getting Around	5
2.1	Quick Start	5
2.2	Keys	7
2.3	Addresses	10
2.4	Scripts	12
2.5	Transactions	15
2.6	SegWit	21
2.7	Taproot	25
2.8	HD Wallets	33
2.9	PSBT	36
2.10	Blocks	43
2.11	Bitcoin Core RPC Proxy	45
2.12	Setup API	47
2.13	Keys and Addresses API	48
2.14	Script API	62
2.15	Transactions API	65
2.16	HD Wallet API	75
2.17	PSBT API	76
2.18	Block API	79
2.19	Utilities API	86
2.20	Proxy API	91
3	Indices and Tables	93

`python-bitcoin-utils` is a pure-Python educational library for building, parsing, and signing Bitcoin data structures. It exposes low-level objects for keys, addresses, scripts, transactions, PSBTs, blocks, HD derivation, and Bitcoin Core RPC calls.

The library is intentionally close to the Bitcoin primitives. You usually build objects directly, inspect their fields, serialize them, and pass them to the next step. This documentation is organized the same way: start with workflows, then use the API reference for exact methods and arguments.

SECURITY MODEL

The private-key code is pure Python and intended for learning, tests, testnet, and offline experimentation. ECDSA signing and Taproot/Schnorr signing are not side-channel hardened. Do not use this library to protect real funds in timing-observable environments.

On mainnet, the library emits a one-time warning when private-key operations are used. Testnet, testnet4, signet, and regtest stay quiet so examples remain usable in teaching material.

GETTING AROUND

2.1 Quick Start

Every script should first select a network:

```
from bitcoinutils.setup import setup

setup("testnet")
```

2.1.1 Keys and Addresses

Create a private key, derive its public key, and render common address types:

```
from bitcoinutils.keys import PrivateKey

private_key = PrivateKey()
public_key = private_key.get_public_key()

print(private_key.to_wif())
print(public_key.to_hex())
print(public_key.get_address().to_string())
print(public_key.get_segwit_address().to_string())
print(public_key.get_taproot_address().to_string())
```

2.1.2 Simple P2PKH Transaction

The complete P2PKH example builds an unsigned transaction, signs one input, and places the signature and public key in the input scriptSig:

```
1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12
```

(continues on next page)

(continued from previous page)

```

13 from bitcoinutils.setup import setup
14 from bitcoinutils.utils import to_satoshis
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput
16 from bitcoinutils.keys import P2pkhAddress, PrivateKey
17 from bitcoinutils.script import Script
18
19
20 def main():
21     # always remember to setup the network
22     setup("testnet")
23
24     # create transaction input from tx id of UTXO (contained 0.4 tBTC)
25     txin = TxInput(
26         "fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0
27     )
28
29     # create transaction output using P2PKH scriptPubKey (locking script)
30     addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
31     txout = TxOutput(
32         to_satoshis(0.1),
33         Script(
34             ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
35         ),
36     )
37
38     # create another output to get the change - remaining 0.01 is tx fees
39     # note that this time we used to_script_pub_key() to create the P2PKH
40     # script
41     change_addr = P2pkhAddress("mmYNBho9BWBQb2dSniP1NjvnPoj5EVWw89w")
42     change_txout = TxOutput(to_satoshis(0.29), change_addr.to_script_pub_key())
43     # change_txout = TxOutput(to_satoshis(0.29), Script(['OP_DUP', 'OP_HASH160',
44     # change_addr.to_hash160(),
45     # 'OP_EQUALVERIFY', 'OP_CHECKSIG'])))
46
47     # create transaction from inputs/outputs -- default locktime is used
48     tx = Transaction([txin], [txout, change_txout])
49
50     # print raw transaction
51     print("\nRaw unsigned transaction:\n" + tx.serialize())
52
53     # use the private key corresponding to the address that contains the
54     # UTXO we are trying to spend to sign the input
55     sk = PrivateKey("cRvyLwCPLU88jsyj94L7iljQX5C2f8koG4G2gevN4BeSGcEvfKe9")
56
57     # note that we pass the scriptPubkey as one of the inputs of sign_input
58     # because it is used to replace the scriptSig of the UTXO we are trying to
59     # spend when creating the transaction digest
60     from_addr = P2pkhAddress("myPAE9HwPeKhh8FjKwBNBaHnemApo3dw6e")
61     sig = sk.sign_input(
62         tx,
63         0,
64         Script(

```

(continues on next page)

(continued from previous page)

```

65         [
66             "OP_DUP",
67             "OP_HASH160",
68             from_addr.to_hash160(),
69             "OP_EQUALVERIFY",
70             "OP_CHECKSIG",
71         ]
72     ),
73 )
74 # print(sig)
75
76 # get public key as hex
77 pk = sk.get_public_key().to_hex()
78
79 # set the scriptSig (unlocking script)
80 txin.script_sig = Script([sig, pk])
81 signed_tx = tx.serialize()
82
83 # print raw signed transaction ready to be broadcasted
84 print("\nRaw signed transaction:\n" + signed_tx)
85
86 # print the size of the final transaction
87 print("\nSigned transaction size (in bytes):\n" + str(tx.get_size()))
88
89 if __name__ == "__main__":
90     main()
91

```

2.1.3 Running Examples From the Checkout

If you run examples directly from the repository, use `PYTHONPATH=.` so Python imports the local checkout instead of an installed package:

```
PYTHONPATH=. venv/bin/python examples/p2pkh_transaction.py
```

2.2 Keys

Key objects are defined in `bitcoinutils.keys`.

2.2.1 Private Keys

`PrivateKey` can be created randomly, from WIF, from raw bytes, or from a secret exponent:

```

from bitcoinutils.setup import setup
from bitcoinutils.keys import PrivateKey

setup("testnet")

random_key = PrivateKey()
wif_key = PrivateKey.from_wif("cRvyLwCPLU88jsyj94L7iJjQX5C2f8koG4G2gevN4BeSGcEvfKe9")
exponent_key = PrivateKey(secret_exponent=1)

```

(continues on next page)

(continued from previous page)

```
print(random_key.to_wif())
print(wif_key.to_bytes().hex())
print(exponent_key.get_public_key().to_hex())
```

2.2.2 Public Keys

PublicKey accepts compressed SEC, uncompressed SEC, or x-only Taproot-style hex strings.

```
from bitcoinutils.keys import PublicKey

pub = PublicKey("0279be667ef9dcbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798")

print(pub.to_hex())
print(pub.to_hex(compressed=False))
print(pub.to_x_only_hex())
print(pub.to_hash160())
```

2.2.3 Message Signing

Bitcoin message signatures use the compact recoverable format. The library can sign and verify them:

```
message = "The test!"
signature = wif_key.sign_message(message)
address = wif_key.get_public_key().get_address().to_string()

assert signature is not None
assert PublicKey.verify_message(address, signature, message)
```

2.2.4 Security Note

Private-key operations are pure Python and are not side-channel hardened. Warnings are emitted on mainnet by default and can be disabled with:

```
from bitcoinutils.setup import set_security_warnings

set_security_warnings(False)
```

2.2.5 Example

```
1 # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2 #
3 # This file is part of python-bitcoin-utils
4 #
5 # It is subject to the license terms in the LICENSE file found in the top-level
6 # directory of this distribution.
7 #
8 # No part of python-bitcoin-utils, including this file, may be copied,
9 # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
```

(continues on next page)

(continued from previous page)

```

12
13 from bitcoinutils.setup import setup
14 from bitcoinutils.keys import PrivateKey, PublicKey
15
16
17 def main():
18     # always remember to setup the network
19     setup("mainnet")
20
21     # create a private key (deterministically)
22     priv = PrivateKey(secret_exponent=1)
23
24     # compressed is the default
25     print("\nPrivate key WIF:", priv.to_wif(compressed=True))
26
27     # could also instantiate from existing WIF key
28     # priv = PrivateKey.from_wif('KwDiBf89qGgbjEhKnhxjUh7LrciVRzI3qYjgd9m7Rfu73SvHnOwn')
29
30     # get the public key
31     pub = priv.get_public_key()
32
33     # compressed is the default
34     print("Public key:", pub.to_hex(compressed=True))
35
36     # get address from public key
37     address = pub.get_address()
38
39     # print the address and hash160 - default is compressed address
40     print("Address:", address.to_string())
41     print("Hash160:", address.to_hash160())
42
43     print("\n-----\n")
44
45     # sign a message with the private key and verify it
46     message = "The test!"
47     signature = priv.sign_message(message)
48     assert signature is not None
49     print("The message to sign:", message)
50     print("The signature is:", signature)
51
52     if PublicKey.verify_message(address.to_string(), signature, message):
53         print("The signature is valid!")
54     else:
55         print("The signature is NOT valid!")
56
57
58 if __name__ == "__main__":
59     main()

```

2.3 Addresses

Address classes live in `bitcoinutils.keys`.

2.3.1 Supported Types

- `P2pkhAddress`: legacy pay-to-public-key-hash.
- `P2shAddress`: pay-to-script-hash.
- `P2wpkhAddress`: native SegWit v0 pay-to-witness-public-key-hash.
- `P2wshAddress`: native SegWit v0 pay-to-witness-script-hash.
- `P2trAddress`: Taproot, SegWit v1.

2.3.2 From Public Keys

```
from bitcoinutils.keys import PrivateKey

priv = PrivateKey()
pub = priv.get_public_key()

legacy = pub.get_address()
segwit = pub.get_segwit_address()
taproot = pub.get_taproot_address()

print(legacy.to_string())
print(segwit.to_string())
print(taproot.to_string())
```

2.3.3 From Scripts

P2SH addresses can be built directly from a redeem script. P2WSH addresses are built from a witness script:

```
from bitcoinutils.keys import P2shAddress, P2wshAddress
from bitcoinutils.script import Script

redeem_script = Script([pub.to_hex(), "OP_CHECKSIG"])

p2sh_addr = P2shAddress.from_script(redeem_script)
p2wsh_addr = P2wshAddress.from_script(redeem_script)

print(p2sh_addr.to_string())
print(p2wsh_addr.to_string())
```

2.3.4 From Existing Address Strings

```
from bitcoinutils.keys import P2pkhAddress, P2wpkhAddress

p2pkh = P2pkhAddress.from_address("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
p2wpkh = P2wpkhAddress.from_address("tb1qxmt9xgewg6mxc4mvnzvrzu4f2v0gy782fydg0w")

print(p2pkh.to_hash160())
print(p2wpkh.to_witness_program())
```

2.3.5 ScriptPubKeys

All address objects expose `to_script_pub_key()`. Use it when constructing a transaction output:

```
from bitcoinutils.transactions import TxOutput
from bitcoinutils.utils import to_satoshis

txout = TxOutput(to_satoshis(0.001), p2pkh.to_script_pub_key())
```

2.3.6 Example

```
1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12
13 from bitcoinutils.setup import setup
14 from bitcoinutils.script import Script
15 from bitcoinutils.keys import P2wpkhAddress, P2wshAddress, P2shAddress, PrivateKey
16
17
18 def main():
19     # always remember to setup the network
20     setup("testnet")
21
22     # could also instantiate from existing WIF key
23     priv = PrivateKey.from_wif("cVdte9ei2xsVjmZSPtyucG43YZgNkmKTqhwiUA8M4Fc3LdPJxPmZ")
24
25     # compressed is the default
26     print("\nPrivate key WIF:", priv.to_wif(compressed=True))
27
28     # get the public key
29     pub = priv.get_public_key()
30
31     # compressed is the default
32     print("Public key:", pub.to_hex(compressed=True))
33
34     # get address from public key
35     address = pub.get_segwit_address()
36
37     # print the address and hash - default is compressed address
38     print("Native Address:", address.to_string())
39     segwit_hash = address.to_witness_program()
40     print("Segwit Hash (witness program):", segwit_hash)
41     print("Segwit Version:", address.get_type())
42
```

(continues on next page)

(continued from previous page)

```

43  # test to_string
44  addr2 = P2wpkhAddress.from_witness_program(segwit_hash)
45  print("Created P2wpkhAddress from Segwit Hash and calculate address:")
46  print("Native Address:", addr2.to_string())
47
48  #
49  # display P2SH-P2WPKH
50  #
51
52  # create segwit address
53  addr3 = (
54      PrivateKey.from_wif("cTmyBsxMQ3vyh4J3jCKYn2Au7AhTKvqeYuxxkinsg6Rz3BBPrYKK")
55      .get_public_key()
56      .get_segwit_address()
57  )
58  # wrap in P2SH address
59  addr4 = P2shAddress.from_script(addr3.to_script_pub_key())
60  print("P2SH(P2WPKH):", addr4.to_string())
61
62  #
63  # display P2WSH
64  #
65  p2wpkh_key = PrivateKey.from_wif(
66      "cNn8itYxAng4xR4eMtrPsrPpDpTdVNuw7Jb6kfhFYZ8DLSZBCg37"
67  )
68  script = Script(
69      ["OP_1", p2wpkh_key.get_public_key().to_hex(), "OP_1", "OP_CHECKMULTISIG"]
70  )
71  p2wsh_addr = P2wshAddress.from_script(script)
72  print("P2WSH of P2PK:", p2wsh_addr.to_string())
73
74  #
75  # display P2SH-P2WSH
76  #
77  p2sh_p2wsh_addr = P2shAddress.from_script(p2wsh_addr.to_script_pub_key())
78  print("P2SH(P2WSH of P2PK):", p2sh_p2wsh_addr.to_string())
79
80
81  if __name__ == "__main__":
82      main()

```

2.4 Scripts

Script represents a Bitcoin script as a list of opcode names, integers, and hex data pushes.

2.4.1 Creating Scripts

```

from bitcoinutils.script import Script

p2pkh_script = Script([
    "OP_DUP",

```

(continues on next page)

(continued from previous page)

```

    "OP_HASH160",
    "751e76e8199196d454941c45d1b3a323f1433bd6",
    "OP_EQUALVERIFY",
    "OP_CHECKSIG",
])

print(p2pkh_script.to_hex())

```

2.4.2 Integers and Data Pushes

Small integers 0 through 16 become OP_0 through OP_16. Larger integers are encoded as script numbers. Hex strings are encoded using the smallest valid pushdata opcode.

```
Script([2, "aa" * 33, "bb" * 33, 2, "OP_CHECKMULTISIG"])
```

2.4.3 Parsing and Copying

```

raw = p2pkh_script.to_hex()
parsed = Script.from_raw(raw)
copied = Script.copy(parsed)

```

2.4.4 ScriptPubKey Helpers

Scripts can be wrapped into P2SH or P2WSH scriptPubKeys:

```

redeem_script = Script(["02" + "11" * 32, "OP_CHECKSIG"])

p2sh_script_pubkey = redeem_script.to_p2sh_script_pub_key()
p2wsh_script_pubkey = redeem_script.to_p2wsh_script_pub_key()

```

2.4.5 Script Type Predicates

The module provides helpers for common output patterns:

```

from bitcoinutils.script import is_p2pkh, is_p2sh, is_p2wpkh, is_p2wsh, is_p2tr

assert is_p2pkh(p2pkh_script)

```

2.4.6 Example

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.

```

(continues on next page)

(continued from previous page)

```

11
12
13 from bitcoinutils.setup import setup
14 from bitcoinutils.utils import to_satoshis
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput
16 from bitcoinutils.keys import P2pkhAddress, PrivateKey
17 from bitcoinutils.script import Script
18
19
20 #
21 # Note that a non-standard transaction can only be included in a block if a
22 # miner agrees with it. For this to work one needs to use a node setup up
23 # for regtest so that you can mine your own blocks; unless you mine your own
24 # testnet/mainnet blocks.
25 # Node's config file requires:
26 #   regtest=1
27 #   acceptnonstdtxn=1
28 #
29 def main():
30     # always remember to setup the network
31     setup("regtest")
32
33     # create transaction input from tx id of UTXO (contained 0.4 tBTC)
34     txin = TxInput(
35         "e2d08a63a540000222d6a92440436375d8b1bc89a2638dc5366833804287c83f", 1
36     )
37
38     # locking script expects 2 numbers that when added equal 5 (silly example)
39     txout = TxOutput(to_satoshis(0.9), Script(["OP_ADD", "OP_5", "OP_EQUAL"]))
40
41     # create another output to get the change - remaining 0.01 is tx fees
42     # note that this time we used to_script_pub_key() to create the P2PKH
43     # script
44     change_addr = P2pkhAddress("mrCDrCybB6J1vRfbwM5hemdJz73FwDBC8r")
45     change_txout = TxOutput(to_satoshis(2), change_addr.to_script_pub_key())
46
47     # create transaction from inputs/outputs -- default locktime is used
48     tx = Transaction([txin], [txout, change_txout])
49
50     # print raw transaction
51     print("\nRaw unsigned transaction:\n" + tx.serialize())
52
53     # use the private key corresponding to the address that contains the
54     # UTXO we are trying to spend to sign the input
55     sk = PrivateKey("cMahea7zqxrtgAbB7LSGbcQUr1uX1ojuat9jZodMN87JcbXMTcA")
56
57     # note that we pass the scriptPubkey as one of the inputs of sign_input
58     # because it is used to replace the scriptSig of the UTXO we are trying to
59     # spend when creating the transaction digest
60     from_addr = P2pkhAddress("mrCDrCybB6J1vRfbwM5hemdJz73FwDBC8r")
61     sig = sk.sign_input(
62         tx,

```

(continues on next page)

(continued from previous page)

```

63     0,
64     Script(
65         [
66             "OP_DUP",
67             "OP_HASH160",
68             from_addr.to_hash160(),
69             "OP_EQUALVERIFY",
70             "OP_CHECKSIG",
71         ]
72     ),
73 )
74 # print(sig)
75
76 # get public key as hex
77 pk = sk.get_public_key()
78 pk = pk.to_hex()
79 # print (pk)
80
81 # set the scriptSig (unlocking script)
82 txin.script_sig = Script([sig, pk])
83 signed_tx = tx.serialize()
84
85 # print raw signed transaction ready to be broadcasted
86 print("\nRaw signed transaction:\n" + signed_tx)
87
88
89 if __name__ == "__main__":
90     main()

```

2.5 Transactions

The transaction module provides low-level transaction objects:

- TxInput references a previous output.
- TxOutput carries an amount in satoshis and a locking script.
- TxWitnessInput stores a witness stack.
- Transaction serializes, parses, hashes, and builds signing digests.
- Sequence and Locktime help construct timelock values.

2.5.1 Constructing a Transaction

```

from bitcoinutils.transactions import Transaction, TxInput, TxOutput
from bitcoinutils.keys import P2pkhAddress
from bitcoinutils.utils import to_satoshis

txin = TxInput("fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0)
txout = TxOutput(to_satoshis(0.1), P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR").to_
    ↪script_pub_key())
tx = Transaction([txin], [txout])

```

(continues on next page)

(continued from previous page)

```
print(tx.serialize())
```

2.5.2 Legacy Signing

For legacy inputs, pass the previous output's scriptPubKey to PrivateKey.sign_input and then place the signature and public key in script_sig.

```
sig = private_key.sign_input(tx, 0, previous_script_pubkey)
txin.script_sig = Script([sig, private_key.get_public_key().to_hex()])
```

2.5.3 SegWit Signing

For SegWit inputs, create the transaction with has_segwit=True, pass the spent amount to sign_segwit_input, and put the signature stack in TxWitnessInput.

```
tx = Transaction([txin], [txout], has_segwit=True)
sig = private_key.sign_segwit_input(tx, 0, script_code, amount)
tx.witnesses.append(TxWitnessInput([sig, private_key.get_public_key().to_hex()])))
```

2.5.4 Parsing Transactions

```
parsed = Transaction.from_raw(tx.serialize())
print(parsed.get_txid())
print(parsed.get_size())
```

2.5.5 Timelocks and RBF

Use Sequence for relative timelocks and replace-by-fee sequences, and Locktime for transaction-level locktime.

```
from bitcoinutils.constants import TYPE_RELATIVE_TIMELOCK
from bitcoinutils.transactions import Sequence

sequence = Sequence(TYPE_RELATIVE_TIMELOCK, 200).for_input_sequence()
txin = TxInput(prev_txid, 0, sequence=sequence)
```

2.5.6 Examples

```
1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
```

(continues on next page)

(continued from previous page)

```

13 from bitcoinutils.setup import setup
14 from bitcoinutils.utils import to_satoshis
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput
16 from bitcoinutils.keys import P2pkhAddress, PrivateKey
17 from bitcoinutils.script import Script
18
19
20 def main():
21     # always remember to setup the network
22     setup("testnet")
23
24     # create transaction input from tx id of UTXO (contained 0.4 tBTC)
25     txin = TxInput(
26         "fb48f4e23bf6ddf606714141ac78c3e921c8c0bebeb7c8abb2c799e9ff96ce6c", 0
27     )
28
29     # create transaction output using P2PKH scriptPubKey (locking script)
30     addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
31     txout = TxOutput(
32         to_satoshis(0.1),
33         Script(
34             ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
35         ),
36     )
37
38     # create another output to get the change - remaining 0.01 is tx fees
39     # note that this time we used to_script_pub_key() to create the P2PKH
40     # script
41     change_addr = P2pkhAddress("mmYNBho9BWBQ2dSniP1NjvnPoj5EVWw89w")
42     change_txout = TxOutput(to_satoshis(0.29), change_addr.to_script_pub_key())
43     # change_txout = TxOutput(to_satoshis(0.29), Script(['OP_DUP', 'OP_HASH160',
44     #                                                     change_addr.to_hash160(),
45     #                                                     'OP_EQUALVERIFY', 'OP_CHECKSIG']))
46
47     # create transaction from inputs/outputs -- default locktime is used
48     tx = Transaction([txin], [txout, change_txout])
49
50     # print raw transaction
51     print("\nRaw unsigned transaction:\n" + tx.serialize())
52
53     # use the private key corresponding to the address that contains the
54     # UTXO we are trying to spend to sign the input
55     sk = PrivateKey("cRvyLwCPLU88jsyj94L7iljQX5C2f8koG4G2gevN4BeSGcEvfKe9")
56
57     # note that we pass the scriptPubkey as one of the inputs of sign_input
58     # because it is used to replace the scriptSig of the UTXO we are trying to
59     # spend when creating the transaction digest
60     from_addr = P2pkhAddress("myPAE9HwPeKhh8FjKwBNBaHnemApo3dw6e")
61     sig = sk.sign_input(
62         tx,
63         0,
64         Script(

```

(continues on next page)

(continued from previous page)

```

65         [
66             "OP_DUP",
67             "OP_HASH160",
68             from_addr.to_hash160(),
69             "OP_EQUALVERIFY",
70             "OP_CHECKSIG",
71         ]
72     ),
73 )
74 # print(sig)
75
76 # get public key as hex
77 pk = sk.get_public_key().to_hex()
78
79 # set the scriptSig (unlocking script)
80 txin.script_sig = Script([sig, pk])
81 signed_tx = tx.serialize()
82
83 # print raw signed transaction ready to be broadcasted
84 print("\nRaw signed transaction:\n" + signed_tx)
85
86 # print the size of the final transaction
87 print("\nSigned transaction size (in bytes):\n" + str(tx.get_size()))
88
89
90 if __name__ == "__main__":
91     main()

```

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12
13 from bitcoinutils.setup import setup
14 from bitcoinutils.utils import to_satoshis
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput
16 from bitcoinutils.keys import P2pkhAddress, PrivateKey
17 from bitcoinutils.script import Script
18 from bitcoinutils.constants import SIGHASH_ALL, SIGHASH_SINGLE, SIGHASH_ANYONECANPAY
19
20
21 def main():
22     # always remember to setup the network
23     setup("testnet")
24

```

(continues on next page)

(continued from previous page)

```

25  # create transaction input from tx id of UTXO (contained 0.39 tBTC)
26  # 0.1 tBTC
27  txin = TxInput(
28      "76464c2b9e2af4d63ef38a77964b3b77e629dddefc5cb9eb1a3645b1608b790f", 0
29  )
30  # 0.29 tBTC
31  txin2 = TxInput(
32      "76464c2b9e2af4d63ef38a77964b3b77e629dddefc5cb9eb1a3645b1608b790f", 1
33  )
34
35  # create transaction output using P2PKH scriptPubKey (locking script)
36  addr = P2pkhAddress("myPAE9HwPeKHh8FjKwBNBaHnemApo3dw6e")
37  txout = TxOutput(
38      to_satoshis(0.3),
39      Script(
40          ["OP_DUP", "OP_HASH160", addr.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
41      ),
42  )
43
44  # create another output to get the change - remaining 0.01 is tx fees
45  change_addr = P2pkhAddress("mmYNBho9BWBQ2dSniP1NJvnPoJ5EVWw89w")
46  change_txout = TxOutput(
47      to_satoshis(0.08),
48      Script(
49          [
50              "OP_DUP",
51              "OP_HASH160",
52              change_addr.to_hash160(),
53              "OP_EQUALVERIFY",
54              "OP_CHECKSIG",
55          ]
56      ),
57  )
58
59  # create transaction from inputs/outputs -- default locktime is used
60  tx = Transaction([txin, txin2], [txout, change_txout])
61
62  # print raw transaction
63  print("\nRaw unsigned transaction:\n" + tx.serialize())
64
65  #
66  # use the private keys corresponding to the addresses that contains the
67  # UTXOs we are trying to spend to create the signatures
68  #
69
70  sk = PrivateKey("cTALNpTpRbbxTCJ2A5Vq88UxT44w1PE2cYqiB3n4hRvzyCev1Wwo")
71  sk2 = PrivateKey("cVf3kGh6552jU2rLaKwXTKq5APHPoZqCP4GQzQirWGHFoHQ9rEVt")
72
73  # we could have derived the addresses from the secret keys
74  from_addr = P2pkhAddress("n4bkvTyU1dVdzsrhWBqBw8fEMbHjJvtmJR")
75  from_addr2 = P2pkhAddress("mmYNBho9BWBQ2dSniP1NJvnPoJ5EVWw89w")
76

```

(continues on next page)

(continued from previous page)

```

77  # sign the first input
78  sig = sk.sign_input(
79      tx,
80      0,
81      Script(
82          [
83              "OP_DUP",
84              "OP_HASH160",
85              from_addr.to_hash160(),
86              "OP_EQUALVERIFY",
87              "OP_CHECKSIG",
88          ]
89      ),
90      SIGHASH_ALL | SIGHASH_ANYONECANPAY,
91  )
92  # print(sig)
93
94  # sign the second input
95  sig2 = sk2.sign_input(
96      tx,
97      1,
98      Script(
99          [
100             "OP_DUP",
101             "OP_HASH160",
102             from_addr2.to_hash160(),
103             "OP_EQUALVERIFY",
104             "OP_CHECKSIG",
105         ]
106     ),
107     SIGHASH_SINGLE | SIGHASH_ANYONECANPAY,
108 )
109 # print(sig2)
110
111 # get public key as hex
112 pk = sk.get_public_key()
113 pk = pk.to_hex()
114 # print (pk)
115
116 # get public key as hex
117 pk2 = sk2.get_public_key()
118 pk2 = pk2.to_hex()
119
120 # set the scriptSig (unlocking script)
121 txin.script_sig = Script([sig, pk])
122 txin2.script_sig = Script([sig2, pk2])
123 signed_tx = tx.serialize()
124
125 # print raw signed transaction ready to be broadcasted
126 print("\nRaw signed transaction:\n" + signed_tx)
127 print("\nTxId:", tx.get_txid())
128

```

(continues on next page)

(continued from previous page)

```

129
130 if __name__ == "__main__":
131     main()

```

2.6 SegWit

SegWit separates witness data from the transaction data used for the legacy transaction id. The library supports native SegWit v0 outputs and nested P2SH-SegWit outputs.

2.6.1 P2WPKH Addresses

```

from bitcoinutils.keys import PrivateKey

pub = PrivateKey().get_public_key()
address = pub.get_segwit_address()
script_pubkey = address.to_script_pub_key()

```

2.6.2 Spending P2WPKH

For P2WPKH, the signing script code is the P2PKH template using the public key hash.

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis
14 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
15 from bitcoinutils.keys import P2pkhAddress, PrivateKey
16 from bitcoinutils.script import Script
17
18
19 def main():
20     # always remember to setup the network
21     setup("testnet")
22
23     # the key that corresponds to the P2WPKH address
24     priv = PrivateKey("cVdte9ei2xsVjmZSPtyucG43YZgNkmKTqhwUA8M4Fc3LdPJxPmZ")
25
26     pub = priv.get_public_key()
27
28     fromAddress = pub.get_segwit_address()
29     print(fromAddress.to_string())
30

```

(continues on next page)

(continued from previous page)

```

31  # amount is needed to sign the segwit input
32  fromAddressAmount = to_satoshis(0.01)
33
34  # UTXO of fromAddress
35  txid = "13d2d30eca974e8fa5da11b9608fa36905a22215e8df895e767fc903889367ff"
36  vout = 0
37
38  toAddress = P2pkhAddress("mrrKUUpJnAjvQntPgZ2Z4kkyr1gbtHmQv28")
39
40  # create transaction input from tx id of UTXO
41  txin = TxInput(txid, vout)
42
43  # in segwit the signature message should commit to the script code
44  # that corresponds to the segwit transaction output
45  # the script code (template) required for signing for p2wpkh is the
46  # same as p2pkh
47  script_code = Script(
48      ["OP_DUP", "OP_HASH160", pub.to_hash160(), "OP_EQUALVERIFY", "OP_CHECKSIG"]
49  )
50
51  # create transaction output
52  txOut = TxOutput(to_satoshis(0.009), toAddress.to_script_pub_key())
53
54  # create transaction without change output - if at least a single input is
55  # segwit we need to set has_segwit=True
56  tx = Transaction([txin], [txOut], has_segwit=True)
57
58  print("\nRaw transaction:\n" + tx.serialize())
59
60  sig = priv.sign_segwit_input(tx, 0, script_code, fromAddressAmount)
61
62  # note that TxWitnessInput gets a list of witness items (not script opcodes)
63  tx.witnesses.append(TxWitnessInput([sig, pub.to_hex()]))
64
65  # print raw signed transaction ready to be broadcasted
66  print("\nRaw signed transaction:\n" + tx.serialize())
67  print("\nTxId:", tx.get_txid())
68
69
70  if __name__ == "__main__":
71      main()

```

2.6.3 Nested P2SH-P2WPKH

Nested SegWit spends use a P2SH scriptSig containing the redeem script and a witness stack containing the signature and public key.

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level

```

(continues on next page)

(continued from previous page)

```

6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis
14 from bitcoinutils.keys import PrivateKey, P2pkhAddress
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
16 from bitcoinutils.script import Script
17
18
19 def main():
20     # always remember to setup the network
21     setup("testnet")
22
23     # the key that corresponds to the P2WPKH address
24     priv = PrivateKey("cNho8fw3bPflKT4jPzpANTsxTsP8aTdVBD6cXksBEXt4KhBN7uVk")
25     pub = priv.get_public_key()
26
27     # the p2sh script and the corresponding address
28     redeem_script = pub.get_segwit_address().to_script_pub_key()
29
30     # the UTXO of the P2SH-P2WPKH that we are trying to spend
31     inp = TxInput("95c5cac558a8b47436a3306ba300c8d7af4cd1d1523d35da3874153c66d99b09", 0)
32
33     # exact amount of UTXO we try to spend
34     amount = 0.0014
35
36     # the address to send funds to
37     to_addr = P2pkhAddress("mvBGdiYC8jLumpJ142ghePYuY8kecQgeqS")
38
39     # the output sending 0.001 -- 0.0004 goes to miners as fee -- no change
40     out = TxOutput(to_satoshis(0.001), to_addr.to_script_pub_key())
41
42     # create a tx with at least one segwit input
43     tx = Transaction([inp], [out], has_segwit=True)
44
45     # script code is the script that is evaluated for a witness program type;
46     # each witness program type has a specific template for the script code;
47     # the script code that corresponds to P2WPKH is the same as P2PKH
48     script_code = pub.get_address().to_script_pub_key()
49
50     # calculate signature using the appropriate script code
51     # remember to include the original amount of the UTXO
52     sig = priv.sign_segwit_input(tx, 0, script_code, to_satoshis(amount))
53
54     # script_sig is the redeem script passed as a single element
55     inp.script_sig = Script([redeem_script.to_hex()])
56
57     # finally, the unlocking script is added as a witness

```

(continues on next page)

(continued from previous page)

```

58     # note that TxWitnessInput gets a list of witness items (not script opcodes)
59     tx.witnesses.append(TxWitnessInput([sig, pub.to_hex()]))
60
61     # print raw signed transaction ready to be broadcasted
62     print("\nRaw signed transaction:\n" + tx.serialize())
63
64
65 if __name__ == "__main__":
66     main()

```

2.6.4 P2WSH

P2WSH spends use the witness script as the signing script and include it as the last witness stack item.

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis
14 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
15 from bitcoinutils.keys import PrivateKey, P2wshAddress, P2wpkhAddress
16 from bitcoinutils.script import Script
17
18
19 def main():
20     # always remember to setup the network
21     setup("testnet")
22
23     priv1 = PrivateKey("cN1XE3ESGgdvr4fWsB7L3BcqXncUauF8Fo8zzv4Sm6WrkiGrxrG")
24     priv2 = PrivateKey("cR8AkcbL2pgBswrHp28AftEznHPPLA86HiTog8MpNCibxwrsUcZ4")
25
26     p2wsh_witness_script = Script(
27         [
28             "OP_1",
29             priv1.get_public_key().to_hex(),
30             priv2.get_public_key().to_hex(),
31             "OP_2",
32             "OP_CHECKMULTISIG",
33         ]
34     )
35
36     fromAddress = P2wshAddress.from_script(p2wsh_witness_script)
37
38     toAddress = P2wpkhAddress.from_address("tb1qtstf97nhk2gycz7vl37esddjpxwt3ut30qp5pn")

```

(continues on next page)

(continued from previous page)

```

39
40 # set values
41 txid = "2042195c40a92353f2ffe30cd0df8d177698560e81807e8bf9174a9c0e98e6c2"
42 vout = 0
43 amount = to_satoshis(0.01)
44
45 # create transaction input from tx id of UTXO
46 txin = TxInput(txid, vout)
47
48 txOut1 = TxOutput(to_satoshis(0.0001), toAddress.to_script_pub_key())
49 txOut2 = TxOutput(to_satoshis(0.0098), fromAddress.to_script_pub_key())
50
51 tx = Transaction([txin], [txOut1, txOut2], has_segwit=True)
52
53 sig1 = priv1.sign_segwit_input(tx, 0, p2wsh_witness_script, amount)
54
55 # note that TxWitnessInput gets a list of witness items (not script opcodes)
56 tx.witnesses.append(TxWitnessInput(["", sig1, p2wsh_witness_script.to_hex()]))
57
58 # print raw signed transaction ready to be broadcasted
59 print("\nRaw signed transaction:\n" + tx.serialize())
60 print("\nTxId:", tx.get_txid())
61
62
63 if __name__ == "__main__":
64     main()

```

2.7 Taproot

Taproot support includes P2TR addresses, key-path signing, script-path signing, Taproot tweaks, and control blocks.

2.7.1 Addresses and Tweaks

`PublicKey.get_taproot_address()` returns a `P2trAddress`. If scripts are passed, the internal key is tweaked with the script tree commitment.

```

1 # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2 #
3 # This file is part of python-bitcoin-utils
4 #
5 # It is subject to the license terms in the LICENSE file found in the top-level
6 # directory of this distribution.
7 #
8 # No part of python-bitcoin-utils, including this file, may be copied,
9 # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.keys import P2trAddress, PrivateKey
14
15

```

(continues on next page)

(continued from previous page)

```

16 def main():
17     # always remember to setup the network
18     setup("testnet")
19
20     # could also instantiate from existing WIF key
21     priv = PrivateKey.from_wif("cRPxBiKrJsH94FLugmiL4xnezMyoFqGcf4kdgNXGuypNERhMK6AT")
22
23     # compressed is the default
24     print("\nPrivate key WIF:", priv.to_wif())
25
26     # get the public key
27     pub = priv.get_public_key()
28
29     # public keys
30     print("Public key as usual:", pub.to_hex())
31     print("Taproot tweaked public key:", pub.to_taproot_hex())
32
33     # get address from public key
34     address = pub.get_taproot_address()
35
36     # print the address and hash - default is compressed address
37     print("Native Address:", address.to_string())
38     taproot_pk = address.to_witness_program()
39     print("Taproot witness program:", taproot_pk)
40     print("Segwit Version:", address.get_type())
41
42     # test to_string
43     addr2 = P2trAddress.from_witness_program(taproot_pk)
44     print("Created P2trAddress from public key and calculate address:")
45     print("Native Address:", addr2.to_string())
46
47     assert (
48         address.to_string()
49         == "tb1pdr8q4tuqqeglxxhxl3trxt0dy5jrnaqvg0ddwu7plraxvntp8dqv8kvyq"
50     )
51     assert (
52         addr2.to_string()
53         == "tb1pdr8q4tuqqeglxxhxl3trxt0dy5jrnaqvg0ddwu7plraxvntp8dqv8kvyq"
54     )
55
56
57 if __name__ == "__main__":
58     main()

```

2.7.2 Key-Path Spending

For a key-path spend, call `PrivateKey.sign_taproot_input` with all input scriptPubKeys and amounts.

```

1 # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2 #
3 # This file is part of python-bitcoin-utils
4 #

```

(continues on next page)

(continued from previous page)

```

5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis
14 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
15 from bitcoinutils.keys import P2pkhAddress, PrivateKey
16
17
18 def main():
19     # always remember to setup the network
20     setup("testnet")
21
22     # the key that corresponds to the P2WPKH address
23     priv = PrivateKey("cV3R88re3AZSBnWhBBNdiCKTfwPMKkYYjdiR13HQzsU7zoRNx7JL")
24
25     pub = priv.get_public_key()
26
27     fromAddress = pub.get_taproot_address()
28     print(fromAddress.to_string())
29
30     # UTXO of fromAddress
31     txid = "7b6412a0eed56338731e83c606f13ebb7a3756b3e4e1dbbe43a7db8d09106e56"
32     vout = 1
33
34     # all amounts are needed to sign a taproot input
35     # (depending on sighash)
36     first_amount = to_satoshis(0.00005)
37     amounts = [first_amount]
38
39     # all scriptPubKeys are needed to sign a taproot input
40     # (depending on sighash) but always of the spend input
41     first_script_pubkey = fromAddress.to_script_pub_key()
42
43     # alternatively:
44     # first_script_pubkey = Script(['OP_1', pub.to_taproot_hex()])
45
46     utxos_script_pubkeys = [first_script_pubkey]
47
48     toAddress = P2pkhAddress("mtVHHCqCECGwiMbMoZe8ayhJHuTdDbYWdJ")
49
50     # create transaction input from tx id of UTXO
51     txin = TxInput(txid, vout)
52
53     # create transaction output
54     txOut = TxOutput(to_satoshis(0.00004), toAddress.to_script_pub_key())
55
56     # create transaction without change output - if at least a single input is

```

(continues on next page)

(continued from previous page)

```

57     # segwit we need to set has_segwit=True
58     tx = Transaction([txin], [txOut], has_segwit=True)
59
60     print("\nRaw transaction:\n" + tx.serialize())
61
62     print("\ntxid: " + tx.get_txid())
63     print("\ntxwid: " + tx.get_wtxid())
64
65     # sign taproot input
66     # to create the digest message to sign in taproot we need to
67     # pass all the utxos' scriptPubKeys and their amounts
68     sig = priv.sign_taproot_input(tx, 0, utxos_script_pubkeys, amounts)
69     # print(sig)
70
71     tx.witnesses.append(TxWitnessInput([sig]))
72
73     # print raw signed transaction ready to be broadcasted
74     print("\nRaw signed transaction:\n" + tx.serialize())
75
76     print("\nTxId:", tx.get_txid())
77     print("\nTxwId:", tx.get_wtxid())
78
79     print("\nSize:", tx.get_size())
80     print("\nvSize:", tx.get_vsize())
81
82
83 if __name__ == "__main__":
84     main()

```

2.7.3 Script-Path Spending

Script-path spends add the executed tapleaf script and a control block to the witness stack.

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis, ControlBlock
14 from bitcoinutils.script import Script
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
16 from bitcoinutils.keys import PrivateKey
17 from bitcoinutils.hdwallet import HDWallet
18
19

```

(continues on next page)

(continued from previous page)

```

20 def main():
21     # always remember to setup the network
22     setup("testnet")
23
24     # Keys are hard-coded in the example for simplicity but it is very bad
25     # practice. Normally you would acquire them from env variables, db, etc
26
27     #####
28     # Construct the input #
29     #####
30
31     # get an HDWallet wrapper object by extended private key and path
32     xprivkey = (
33         "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
34         "tpe84f3cVcF7mZZ7mPkntaFXLGJD2tS7YJkWU"
35     )
36     path = "m/86'/1'/0'/0/7"
37     hdw = HDWallet(xprivkey, path)
38     internal_priv = hdw.get_private_key()
39     print("From Private key:", internal_priv.to_wif())
40     internal_pub = internal_priv.get_public_key()
41     print("From Public key:", internal_pub.to_hex())
42
43     # taproot script is a simple P2PK with the following keys
44     privkey_tr_script = PrivateKey(
45         "cQwzrJyTNWbEwhPEmQ3Qoo4jSfHdHEtdbL4kNBgHUKhirczcQw7G"
46     )
47     pubkey_tr_script = privkey_tr_script.get_public_key()
48     tr_script_p2pk = Script([pubkey_tr_script.to_x_only_hex(), "OP_CHECKSIG"])
49
50     # taproot script path address
51     from_address = internal_pub.get_taproot_address([[tr_script_p2pk]])
52     print("From Taproot script address", from_address.to_string())
53
54     # UTXO of from_address from pubkey 02
55     txid = "348f577ae2509b3b73ebd810c3cdcb18045ef62b43378aed283b3259afe493b1"
56     vout = 0
57
58     # create transaction input from tx id of UTXO
59     tx_in = TxInput(txid, vout)
60
61     # all amounts are needed to sign a taproot input
62     # (depending on sighash)
63     amount = to_satoshis(0.00009)
64     amounts = [amount]
65
66     # all scriptPubKeys (in hex) are needed to sign a taproot input
67     # (depending on sighash but always of the spend input)
68     scriptPubkey = from_address.to_script_pub_key()
69     utxos_scriptPubkeys = [scriptPubkey]
70
71     #####

```

(continues on next page)

(continued from previous page)

```

72  # Construct the output #
73  #####
74
75  hdw.from_path("m/86'/1'/0'/0/5")
76  to_priv = hdw.get_private_key()
77  print("To Private key:", to_priv.to_wif())
78
79  to_pub = to_priv.get_public_key()
80  print("To Public key:", to_pub.to_hex())
81
82  # taproot key path address
83  to_address = to_pub.get_taproot_address()
84  print("To Taproot address:", to_address.to_string())
85
86  # create transaction output
87  tx_out = TxOutput(to_satoshis(0.000085), to_address.to_script_pub_key())
88
89  # create transaction without change output - if at least a single input is
90  # segwit we need to set has_segwit=True
91  tx = Transaction([tx_in], [tx_out], has_segwit=True)
92
93  print("\nRaw transaction:\n" + tx.serialize())
94
95  print("\ntxid: " + tx.get_txid())
96  print("\ntxwid: " + tx.get_wtxid())
97
98  # sign taproot input
99  # to create the digest message to sign in taproot we need to
100 # pass all the utxos' scriptPubKeys, their amounts and taproot script
101 # we sign with the private key corresponding to the script - no keys
102 # tweaking required
103 sig = privkey_tr_script.sign_taproot_input(
104     tx,
105     0,
106     utxos_scriptPubkeys,
107     amounts,
108     script_path=True,
109     tapleaf_script=tr_script_p2pk,
110     tweak=False,
111 )
112
113 control_block = ControlBlock(internal_pub, scripts=[tr_script_p2pk], index=0, is_
↪ odd=to_address.is_odd())
114
115 tx.witnesses.append(
116     TxWitnessInput([sig, tr_script_p2pk.to_hex(), control_block.to_hex()])
117 )
118
119 # print raw signed transaction ready to be broadcasted
120 print("\nRaw signed transaction:\n" + tx.serialize())
121
122 print("\nTxId:", tx.get_txid())

```

(continues on next page)

(continued from previous page)

```

123     print("\nTxwId:", tx.get_wtxid())
124
125     print("\nSize:", tx.get_size())
126     print("\nvSize:", tx.get_vsize())
127
128
129 if __name__ == "__main__":
130     main()

```

2.7.4 Multiple Script Paths

The examples directory contains two-, three-, and four-leaf Taproot trees.

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 from bitcoinutils.setup import setup
13 from bitcoinutils.utils import to_satoshis
14 from bitcoinutils.script import Script
15 from bitcoinutils.transactions import Transaction, TxInput, TxOutput, TxWitnessInput
16 from bitcoinutils.keys import PrivateKey
17 from bitcoinutils.hdwallet import HDWallet
18
19
20 def main():
21     # always remember to setup the network
22     setup("testnet")
23
24     #####
25     # Construct the input #
26     #####
27
28     # get an HDWallet wrapper object by extended private key and path
29     xprivkey = (
30         "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
31         "tpe84f3cVcF7mZZ7mPkntaFXLGD2tS7YJkWU"
32     )
33     path = "m/86'/1'/0'/0/5"
34     hdw = HDWallet(xprivkey, path)
35     internal_priv = hdw.get_private_key()
36     print("From Private key:", internal_priv.to_wif())
37
38     internal_pub = internal_priv.get_public_key()
39     print("From Public key:", internal_pub.to_hex())

```

(continues on next page)

(continued from previous page)

```

40 from_address = internal_pub.get_taproot_address()
41 print("From Taproot address:", from_address.to_string())
42
43 # UTXO of fromAddress
44 txid = "29afd65f1aeeab4e4d655b148776fe0097acc617492b0c3f3950b6a95be20f39"
45 vout = 0
46
47 # create transaction input from tx id of UTXO
48 tx_in = TxInput(txid, vout)
49
50 # all amounts are needed to sign a taproot input
51 # (depending on sighash)
52 amount = to_satoshis(0.00004)
53 amounts = [amount]
54
55 # all scriptPubKeys (in hex) are needed to sign a taproot input
56 # (depending on sighash but always of the spend input)
57 scriptPubkey = from_address.to_script_pub_key()
58 utxos_scriptPubkeys = [scriptPubkey]
59
60 #####
61 # Construct the output #
62 #####
63
64 hdw.from_path("m/86'/1'/0'/0/7")
65 to_priv = hdw.get_private_key()
66 print("Send to Private key", to_priv.to_wif())
67 to_pub = to_priv.get_public_key()
68
69 # taproot script A is a simple P2PK with the following keys
70 privkey_tr_script_A = PrivateKey(
71     "cSW2kQbqC9zkqagw8oTYKFTozKuZ214zd6CMTDs4V32cMfH3dgKa"
72 )
73 pubkey_tr_script_A = privkey_tr_script_A.get_public_key()
74 tr_script_p2pk_A = Script([pubkey_tr_script_A.to_x_only_hex(), "OP_CHECKSIG"])
75
76 # taproot script B is a simple P2PK with the following keys
77 privkey_tr_script_B = PrivateKey(
78     "cSv48xapaqy7fPs8VvoSnxNBNA2jpjcuURRqUENu3WVq6Eh4U3JU"
79 )
80 pubkey_tr_script_B = privkey_tr_script_B.get_public_key()
81 tr_script_p2pk_B = Script([pubkey_tr_script_B.to_x_only_hex(), "OP_CHECKSIG"])
82
83 # taproot script C is a simple P2PK with the following keys
84 privkey_tr_script_C = PrivateKey(
85     "cRkZPNnn3jdr64o3PDxNHG68eowDfuCdcyL6nVL4n3czvunuvryC"
86 )
87 pubkey_tr_script_C = privkey_tr_script_C.get_public_key()
88 tr_script_p2pk_C = Script([pubkey_tr_script_C.to_x_only_hex(), "OP_CHECKSIG"])
89
90 # tapleafs in order
91

```

(continues on next page)

(continued from previous page)

```

92  #
93  #
94  #
95  #
96  #
97  #
98  #
99  all_leafs = [[tr_script_p2pk_A, tr_script_p2pk_B], tr_script_p2pk_C]
100
101  # taproot script path address
102  to_address = to_pub.get_taproot_address(all_leafs)
103  print("To Taproot script address", to_address.to_string())
104
105  # create transaction output
106  tx_out = TxOutput(to_satoshis(0.000035), to_address.to_script_pub_key())
107
108  # create transaction without change output - if at least a single input is
109  # segwit we need to set has_segwit=True
110  tx = Transaction([tx_in], [tx_out], has_segwit=True)
111
112  print("\nRaw transaction:\n" + tx.serialize())
113
114  print("\ntxid: " + tx.get_txid())
115  print("\ntxwid: " + tx.get_wtxid())
116
117  # sign taproot input
118  # to create the digest message to sign in taproot we need to
119  # pass all the utxos' scriptPubKeys and their amounts
120  sig = internal_priv.sign_taproot_input(tx, 0, utxos_scriptPubkeys, amounts)
121
122  tx.witnesses.append(TxWitnessInput([sig]))
123
124  # print raw signed transaction ready to be broadcasted
125  print("\nRaw signed transaction:\n" + tx.serialize())
126
127  print("\nTxId:", tx.get_txid())
128  print("\nTxwId:", tx.get_wtxid())
129
130  print("\nSize:", tx.get_size())
131  print("\nvSize:", tx.get_vsize())
132
133
134  if __name__ == "__main__":
135      main()

```

2.8 HD Wallets

bitcoinutils.hdwallet.HDWallet implements the small BIP-32/BIP-39 subset used by the examples: derive private keys from a mnemonic or an extended private key and return them as PrivateKey objects.

2.8.1 From a Mnemonic

```
from bitcoinutils.hdwallet import HDWallet

mnemonic = "addict weather world sense idle purity rich wagon ankle fall cheese spatial"
wallet = HDWallet.from_mnemonic(mnemonic)
wallet.from_path("m/84'/1'/0'/0/0")

private_key = wallet.get_private_key()
print(private_key.get_public_key().get_segwit_address().to_string())
```

2.8.2 From an Extended Private Key

```
wallet = HDWallet.from_xprivate_key(tprv, "m/86'/1'/0'/0/0")
print(wallet.get_private_key().get_public_key().get_taproot_address().to_string())
```

2.8.3 Paths

Only absolute paths starting with `m/` are supported. Hardened components may use `'`, `h`, or `H`.

Common testnet examples:

- `m/44'/1'/0'/0/0` for legacy P2PKH intent.
- `m/84'/1'/0'/0/0` for native SegWit P2WPKH intent.
- `m/86'/1'/0'/0/0` for Taproot P2TR intent.

2.8.4 Example

```
1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12 """Detailed HD wallet examples.
13
14 This example demonstrates the minimal HD functionality provided by
15 bitcoinutils:
16
17 * create a wallet from a BIP-39 mnemonic
18 * derive private keys from different BIP-style paths
19 * derive legacy, segwit v0, and taproot addresses from the same key
20 * switch paths on the same HDWallet instance
21 * create a wallet from an extended private key
22
23 The mnemonic and extended private keys below are for testnet examples only.
24 Do not use them with real funds.
```

(continues on next page)

(continued from previous page)

```

25 """
26
27 from bitcoinutils.setup import setup
28 from bitcoinutils.hdwallet import HDWallet
29
30
31 def print_key_details(title, hdw):
32     """Print the currently selected key and common address types."""
33
34     privkey = hdw.get_private_key()
35     pubkey = privkey.get_public_key()
36
37     print(f"\n{title}")
38     print("-" * len(title))
39     print("WIF:", privkey.to_wif())
40     print("Public key:", pubkey.to_hex())
41     print("Legacy P2PKH:", pubkey.get_address().to_string())
42     print("Native Segwit P2WPKH:", pubkey.get_segwit_address().to_string())
43     print("Taproot P2TR:", pubkey.get_taproot_address().to_string())
44
45
46 def derive_and_print(hdw, path):
47     """Derive an absolute path and print the resulting key/address details."""
48
49     hdw.from_path(path)
50     print_key_details(f"Path {path}", hdw)
51
52
53 def main():
54     setup("testnet")
55
56     mnemonic = (
57         "addict weather world sense idle purity rich wagon "
58         "ankle fall cheese spatial"
59     )
60
61     print("Mnemonic wallet")
62     print("=====")
63     print("Mnemonic:", mnemonic)
64
65     hdw_from_mnemonic = HDWallet(mnemonic=mnemonic)
66
67     # BIP-44: legacy P2PKH-style account path.
68     derive_and_print(hdw_from_mnemonic, "m/44'/1'/0'/0'")
69     derive_and_print(hdw_from_mnemonic, "m/44'/1'/0'/0'1")
70
71     # BIP-84: native segwit account path. The private/public key can still be
72     # rendered as multiple address types; the path communicates wallet intent.
73     derive_and_print(hdw_from_mnemonic, "m/84'/1'/0'/0'")
74
75     # BIP-86: taproot account path.
76     derive_and_print(hdw_from_mnemonic, "m/86'/1'/0'/0'")

```

(continues on next page)

(continued from previous page)

```

77
78 print("\nSwitching paths")
79 print("=====")
80 hdw_from_mnemonic.from_path("m/44'/1'/0'/0/3")
81 addr_a = hdw_from_mnemonic.get_private_key().get_public_key().get_address()
82 print("m/44'/1'/0'/0/3 legacy address:", addr_a.to_string())
83
84 hdw_from_mnemonic.from_path("m/44'/1'/0'/0/4")
85 addr_b = hdw_from_mnemonic.get_private_key().get_public_key().get_address()
86 print("m/44'/1'/0'/0/4 legacy address:", addr_b.to_string())
87
88 xprivkey = (
89     "tprv8ZgxMBicQKsPdQR9RuHpGGxSnNq8Jr3X4WnT6Nf2eq7FajuXyBep5KWYpYEixxx5XdTm1N"
90     "tpe84f3cVcF7mZZ7mPkntaFXLGD2tS7YJkWU"
91 )
92
93 print("\nExtended private key wallet")
94 print("=====")
95 print("Extended private key:", xprivkey)
96
97 hdw_from_xpriv = HDWallet.from_xprivate_key(xprivkey, "m/86'/1'/0'/0/1")
98 print_key_details("Path m/86'/1'/0'/0/1", hdw_from_xpriv)
99
100 hdw_from_xpriv.from_path("m/86'/1'/0'/0/5")
101 print_key_details("Path m/86'/1'/0'/0/5", hdw_from_xpriv)
102
103
104 if __name__ == "__main__":
105     main()

```

2.9 PSBT

The `bitcoinutils.psbt` module implements BIP-174 PSBT v0 workflows.

2.9.1 Roles

The PSBT class can act as:

- Creator: wrap an unsigned Transaction.
- Updater: attach UTXO and script metadata with `update_input`.
- Signer: call `sign_input` with a `PrivateKey`.
- Combiner: merge compatible PSBTs with `combine`.
- Finalizer: call `finalize` or `finalize_input`.
- Extractor: call `extract_transaction`.

2.9.2 Minimal Flow

```
psbt = PSBT(tx)
psbt.update_input(0, non_witness_utxo=previous_tx, redeem_script=redeem_script)
psbt.sign_input(0, private_key)
psbt.finalize()
final_tx = psbt.extract_transaction()
```

2.9.3 2-of-3 Multisig Walkthrough

The examples include a complete 2-of-3 P2SH multisig PSBT workflow split across creator and signer scripts.

PSBT 2-of-3 Multisig Signing Guide

This document describes how to use BIP-174 PSBTs to coordinate a **2-of-3 P2SH multisig** spend across two independent signers. This is one of the most common real-world PSBT use cases: a transaction that requires two out of three keyholders to approve before it can be broadcast.

We simulate the workflow using three scripts (in `examples/``) that each execute one or more BIP-174 roles. Both signing scripts run on the same machine but use different private keys, just as they would if two keyholders were signing on separate devices or in separate locations.

```
```bash
python examples/psbt_2of3_create.py # Step 1: Create unsigned PSBT
python examples/psbt_2of3_sign1.py # Step 2: Signer 1 adds partial signature
python examples/psbt_2of3_sign2.py # Step 3: Signer 2 signs, combines, finalizes,
 extracts
```
```

Background

What is 2-of-3 multisig?

A 2-of-3 multisig address is controlled by three public keys, and any two of the three corresponding private keys are required to spend funds from it. The locking script (redeemScript) looks like:

```
```
OP_2 <pubkey_1> <pubkey_2> <pubkey_3> OP_3 OP_CHECKMULTISIG
```
```

The address itself is the hash of this redeemScript, wrapped in P2SH format (`3...` on mainnet, `2...` on testnet).

Why PSBT?

Without PSBT, coordinating multiple signers requires passing around raw transaction bytes and manually managing partial signatures. PSBT solves this by defining a standard container that carries:

- The unsigned transaction

(continues on next page)

(continued from previous page)

- UTXO data needed for signing
- Script metadata (redeemScript, witnessScript)
- Partial signatures from each signer

Each participant adds their piece and passes the PSBT along. No signer needs to understand what the others are doing; they just sign and pass it on.

BIP-174 roles in this workflow

BIP-174 defines six roles. In our 2-of-3 scenario, they map to three scripts:

| Role | Responsibility | Script |
|----------------------|--|------------------------------------|
| --- | --- | --- |
| **Creator** | Builds the unsigned transaction and wraps it in a PSBT | <code>`psbt_2of3_create.py`</code> |
| **Updater** | Adds the UTXO info, redeemScript, and sighash type | <code>`psbt_2of3_create.py`</code> |
| **Signer 1** | Adds a partial signature using private key A | <code>`psbt_2of3_sign1.py`</code> |
| **Signer 2** | Adds a partial signature using private key B | <code>`psbt_2of3_sign2.py`</code> |
| **Combiner** | Merges the two signed PSBTs into one | <code>`psbt_2of3_sign2.py`</code> |
| **Finalizer** | Constructs the final scriptSig from partial signatures | <code>`psbt_2of3_sign2.py`</code> |
| **Extractor** | Pulls out the fully signed transaction for broadcast | <code>`psbt_2of3_sign2.py`</code> |

Note: the Creator and Updater roles are combined in the first script, and the Combiner/Finalizer/Extractor are combined in the second signer's script. This is a common pattern - in practice the last signer usually finalizes and extracts.

Setup

Shared knowledge (all participants know this)

Before the workflow starts, all three keyholders have agreed on:

1. ****The three public keys**** that form the multisig (and their order in the redeemScript).
2. ****The redeemScript**** itself, derived from those three public keys.
3. ****The P2SH address**** derived from the redeemScript - this is where funds are locked.

Private key distribution

- ****Key A**** - held by Signer 1 only
- ****Key B**** - held by Signer 2 only
- ****Key C**** - held by a third party (cold storage, escrow, etc.) - not used in this spend

Any two of the three can sign. In this example, Signers 1 and 2 cooperate.

Workflow

The PSBT flows through the scripts as a ****base64-encoded string****. In production this string would be passed via file, QR code, NFC, or a coordination server. Here we write

(continues on next page)

(continued from previous page)

→ it to `.psbt` files on disk.

```

73
74
75         psbt_2of3_create.py
76         (Creator + Updater)
77         |
78         v
79         unsigned.psbt          <-- base64 PSBT with UTXO + redeemScript
80         |
81         +-----+-----+
82         |               |
83         v               v
84     psbt_2of3_sign1.py   (copy of unsigned.psbt)
85     (Signer 1)           |
86         |               |
87         v               |
88     signer1.psbt         |
89     (has sig from Key A) |
90         |               |
91         +-----+-----+
92         |               |
93         v               |
94     psbt_2of3_sign2.py   |
95     (Signer 2 + Combiner +
96      Finalizer + Extractor)
97         |               |
98         v               |
99     final_tx.hex         <-- raw signed transaction, ready to broadcast
100
101

```

Step 1: Create the PSBT (`psbt_2of3_create.py`)

This script acts as both **Creator** and **Updater**.

Creator responsibilities:

- Build a `Transaction` with the desired inputs (UTXOs locked in the multisig address) and outputs (where the funds are going).
- Wrap it in a `PSBT` object. The constructor automatically strips scriptSigs and segwit data, producing a clean unsigned transaction.

Updater responsibilities:

- Attach the **non-witness UTXO** (the full previous transaction) for each input. This lets signers verify the amount being spent.
- Attach the **redeemScript** for each input. Without it, signers cannot determine what they are signing for.

```

113
114 ``python
115 # Pseudocode for psbt_2of3_create.py
116
117 setup("testnet")
118
119 # 1. Define the 3 public keys and build the redeemScript

```

(continues on next page)

(continued from previous page)

```

120 pk1 = PublicKey(...)
121 pk2 = PublicKey(...)
122 pk3 = PublicKey(...)
123
124 redeem_script = Script([
125     "OP_2",
126     pk1.to_hex(), pk2.to_hex(), pk3.to_hex(),
127     "OP_3", "OP_CHECKMULTISIG"
128 ])
129
130 # 2. Create the unsigned transaction (Creator)
131 txin = TxInput(prev_txid, vout_index)
132 txout = TxOutput(amount, destination_script_pubkey)
133 tx = Transaction([txin], [txout])
134
135 # 3. Wrap in PSBT
136 psbt = PSBT(tx)
137
138 # 4. Update with UTXO and redeemScript (Updater)
139 psbt.update_input(0,
140     non_witness_utxo=prev_tx,          # full previous transaction
141     redeem_script=redeem_script        # so signers know the multisig structure
142 )
143
144 # 5. Export
145 with open("unsigned.psb", "w") as f:
146     f.write(psbt.to_base64())
147
148
149 **What the PSBT contains at this point:**
150 - Global: the unsigned transaction
151 - Input 0: `non_witness_utxo` (full prev tx) + `redeem_script` (the 2-of-3 script)
152 - Input 0: no signatures yet
153
154 ### Step 2: Signer 1 signs (`psbt_2of3_sign1.py`)
155
156 This script acts as **Signer 1** only.
157
158 **Signer responsibilities (per BIP-174):**
159 - Parse the PSBT.
160 - Verify the UTXO data is consistent (txid of `non_witness_utxo` matches the input's
161   ↪referenced txid).
162 - Produce a partial signature using their private key.
163 - Add the signature to the PSBT as a `partial_sig` entry.
164 - Export the updated PSBT.
165
166 ```python
167 # Pseudocode for psbt_2of3_sign1.py
168
169 setup("testnet")
170
171 # 1. Load the unsigned PSBT

```

(continues on next page)

(continued from previous page)

```

171 with open("unsigned.psbtx") as f:
172     psbt = PSBT.from_base64(f.read().strip())
173
174 # 2. Load Signer 1's private key
175 sk1 = PrivateKey("cXXXX...") # Key A (WIF format)
176
177 # 3. Sign input 0
178 psbt.sign_input(0, sk1)
179
180 # 4. Export - DO NOT finalize (need Signer 2's signature too)
181 with open("signer1.psbtx", "w") as f:
182     f.write(psbt.to_base64())
183
184
185 **What the PSBT contains at this point:**
186 - Everything from before, plus:
187 - Input 0: `partial_sigs` now has one entry: `{pubkey_A: signature_A}`
188
189 **Important:** Signer 1 must NOT finalize the PSBT. Finalization locks in the scriptSig_
190 ↪ and clears partial signatures. Since we need a second signature, the PSBT must remain_
191 ↪ in its partially-signed state.
192
193 ### Step 3: Signer 2 signs, combines, finalizes, and extracts (`psbt_2of3_sign2.py`)
194
195 This script acts as **Signer 2**, **Combiner**, **Finalizer**, and **Extractor**.
196
197 ```python
198 # Pseudocode for psbt_2of3_sign2.py
199
200 setup("testnet")
201
202 # 1. Load both PSBTs
203 with open("unsigned.psbtx") as f:
204     psbt_unsigned = PSBT.from_base64(f.read().strip())
205
206 with open("signer1.psbtx") as f:
207     psbt_signed1 = PSBT.from_base64(f.read().strip())
208
209 # 2. Sign the unsigned copy with Signer 2's key (Signer)
210 sk2 = PrivateKey("cYYYY...") # Key B (WIF format)
211 psbt_unsigned.sign_input(0, sk2)
212 # psbt_unsigned now has: partial_sigs = {pubkey_B: signature_B}
213
214 # 3. Combine the two PSBTs (Combiner)
215 combined = psbt_signed1.combine(psbt_unsigned)
216 # combined now has: partial_sigs = {pubkey_A: sig_A, pubkey_B: sig_B}
217
218 # 4. Finalize (Finalizer)
219 combined.finalize()
220 # This constructs the final scriptSig:
221 #   OP_0 <sig_A> <sig_B> <redeemScript>
222 # and clears partial_sigs, redeem_script, etc.

```

(continues on next page)

(continued from previous page)

```

221
222 # 5. Extract the signed transaction (Extractor)
223 final_tx = combined.extract_transaction()
224 tx_hex = final_tx.serialize()
225
226 with open("final_tx.hex", "w") as f:
227     f.write(tx_hex)
228
229 print(f"Transaction ready to broadcast: {tx_hex}")
230 ```
231
232 **What happens during finalization:**
233
234 The finalize() method detects that the input is a P2SH multisig (by inspecting the
    ↳ redeemScript). It:
235 1. Collects the partial signatures from partial_sigs.
236 2. Orders them to match the pubkey order in the redeemScript (Bitcoin's OP_
    ↳ CHECKMULTISIG requires this).
237 3. Builds the scriptSig: OP_0 <sig_1> <sig_2> <serialized_redeemScript>.
238 4. Clears all non-final fields (partial_sigs, redeem_script, bip32_derivs,
    ↳ sighash_type).
239
240 The extract_transaction() method takes the finalized scriptSig and places it into a
    ↳ standard Bitcoin transaction that can be serialized and broadcast.
241
242 ## Alternative flow: sequential signing
243
244 Instead of combining two separately-signed PSBTs, Signer 2 can sign the PSBT that Signer
    ↳ 1 already signed:
245
246 ```
247 unsigned.psbt --> Signer 1 signs --> signer1.psbt --> Signer 2 signs -->
    ↳ finalize + extract
248 ```
249
250 In this flow, Signer 2's script is simpler:
251
252 ```python
253 # Load Signer 1's output directly
254 with open("signer1.psbt") as f:
255     psbt = PSBT.from_base64(f.read().strip())
256
257 # Sign on top of it (adds a second partial_sig)
258 sk2 = PrivateKey("cYYYYY...")
259 psbt.sign_input(0, sk2)
260 # psbt now has: partial_sigs = {pubkey_A: sig_A, pubkey_B: sig_B}
261
262 # Finalize and extract (no combine needed)
263 psbt.finalize()
264 final_tx = psbt.extract_transaction()
265 ```

```

(continues on next page)

(continued from previous page)

```

267 This is simpler but requires Signer 2 to wait for Signer 1. The combine-based flow
    ↳ allows both signers to work in parallel.
268
269 ## File summary
270
271 | File | BIP-174 Roles | Input | Output |
272 |---|---|---|---|
273 | `psbt_2of3_create.py` | Creator + Updater | UTXO details, 3 public keys | `unsigned.
    ↳ psbt` |
274 | `psbt_2of3_sign1.py` | Signer | `unsigned.psbt`, Key A | `signer1.psbt` |
275 | `psbt_2of3_sign2.py` | Signer + Combiner + Finalizer + Extractor | `unsigned.psbt`,
    ↳ `signer1.psbt`, Key B | `final_tx.hex` |
276
277 ## Key points
278
279 - **Never finalize early.** Only finalize after all required signatures (2 of 3) are
    ↳ present. Finalizing with fewer signatures produces an invalid transaction.
280 - **Public key order matters.** `OP_CHECKMULTISIG` expects signatures in the same order
    ↳ as the public keys in the redeemScript. The PSBT finalizer handles this automatically
    ↳ by matching pubkeys from `partial_sigs` against the redeemScript.
281 - **The PSBT is not confidential.** Anyone who sees the PSBT can see the transaction
    ↳ details and public keys. However, they cannot forge signatures without the private
    ↳ keys.
282 - **The redeemScript must be attached.** Without it in the PSBT, signers cannot
    ↳ determine what type of script they are signing for, and the finalizer cannot construct
    ↳ the correct scriptSig.
283 - **UTXO data is mandatory.** The `non_witness_utxo` (full previous transaction) lets
    ↳ each signer independently verify the amount being spent, preventing fee-manipulation
    ↳ attacks.
284
285 ## Extending to P2SH-P2WSH (segwit multisig)
286
287 For a segwit-wrapped multisig (P2SH-P2WSH), the flow is identical except:
288
289 1. The `update_input` call also includes a `witness_script` (the raw multisig script)
    ↳ and a `witness_utxo` (the specific output being spent).
290 2. The `redeem_script` becomes `OP_0 <SHA256(witness_script)>` - a P2WSH wrapper.
291 3. Finalization produces both a scriptSig (pushing the redeemScript) and a witness stack
    ↳ (`OP_0 <sig_1> <sig_2> <witness_script>`).
292
293 The PSBT library handles these differences transparently. The signing scripts are nearly
    ↳ identical; only the `update_input` call changes.

```

2.10 Blocks

The block module parses Bitcoin block headers and full raw blocks.

2.10.1 Block Headers

`BlockHeader.from_raw` accepts 80-byte header data as hex or bytes.

```
from bitcoinutils.block import BlockHeader

header = BlockHeader.from_raw(raw_header_hex)
print(header.get_block_hash())
print(header.format_timestamp())
print(header.get_target_hex())
```

2.10.2 Blocks

`Block.from_raw` parses magic bytes, size, header, transaction count, and all transactions.

```
from bitcoinutils.block import Block

block = Block.from_raw(raw_block_hex)
print(block.get_transactions_count())
print(block.get_coinbase_transaction())
```

2.10.3 Example

```
1  import os
2  import struct
3  import sys
4
5  # Dynamically add the root path to sys.path
6  root_path = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
7  if root_path not in sys.path:
8      sys.path.append(root_path)
9
10 from bitcoinutils.block import Block
11 from bitcoinutils.constants import BLOCK_MAGIC_NUMBER
12
13 def main():
14     if len(sys.argv) != 2:
15         print("Usage: python script.py <path_to_blkNNNNN.dat>")
16         return
17
18     filename = sys.argv[1]
19
20     try:
21         with open(filename, 'rb') as file:
22             block_height = 0
23             while True:
24                 # Read 8 bytes: 4 for magic number and 4 for block size
25                 preamble = file.read(8)
26                 if len(preamble) < 8:
27                     print("End of file reached.")
28                     break
29
30                 magic, size = struct.unpack('<4sI', preamble)
```

(continues on next page)

(continued from previous page)

```

31         magic_hex = magic.hex()
32
33         if magic_hex not in BLOCK_MAGIC_NUMBER:
34             raise ValueError(f"Unknown or unsupported network magic number:
↪ {magic_hex}")
35
36         network = BLOCK_MAGIC_NUMBER[magic_hex]
37         print(f"\n[{network}] Block #{block_height}")
38
39         block_data = file.read(size)
40         if len(block_data) < size:
41             print("Warning: Truncated block data.")
42             break
43
44         raw_block = preamble + block_data
45         block = Block.from_raw(raw_block)
46
47         # Output block info
48         print("Block Hash:", block.get_block_header().get_block_hash())
49         print("Transactions:", block.get_transactions_count())
50
51         block_height += 1
52
53     except FileNotFoundError:
54         print(f"Error: File '{filename}' not found.")
55     except Exception as e:
56         print(f"An error occurred: {e}")
57
58 if __name__ == '__main__':
59     main()

```

2.11 Bitcoin Core RPC Proxy

NodeProxy is a small wrapper around Bitcoin Core JSON-RPC. It is useful for examples that need to query a local node or broadcast a raw transaction.

2.11.1 Basic Use

```

from bitcoinutils.proxy import NodeProxy

proxy = NodeProxy("bitcoinrpc", "password", host="127.0.0.1", port=18443)
block_count = proxy.getblockcount()
print(block_count)

```

2.11.2 Method Calls

RPC methods are exposed dynamically. Calling `proxy.getblockcount()` maps to the Bitcoin Core RPC method `getblockcount`.

2.11.3 Example

```

1  # Copyright (C) 2018-2025 The python-bitcoin-utils developers
2  #
3  # This file is part of python-bitcoin-utils
4  #
5  # It is subject to the license terms in the LICENSE file found in the top-level
6  # directory of this distribution.
7  #
8  # No part of python-bitcoin-utils, including this file, may be copied,
9  # modified, propagated, or distributed except according to the terms contained
10 # in the LICENSE file.
11
12
13 from bitcoinutils.setup import setup
14 from bitcoinutils.proxy import NodeProxy
15
16
17 def main():
18     # always remember to setup the network
19     setup("regtest")
20
21     # get a node proxy using default host and port
22     proxy = NodeProxy("rpcuser", "rpcpw")
23
24     # call the node's getblockcount JSON-RPC method
25     count = proxy.getblockcount()
26
27     # call the node's getblockhash JSON-RPC method
28     block_hash = proxy.getblockhash(count)
29
30     # call the node's getblock JSON-RPC method and print result
31     block = proxy.getblock(block_hash)
32
33     print("--- Block Information ---")
34     print(f"Height: {block['height']}")
35     print(f"Hash: {block['hash']}")
36     print(f"Transactions: {len(block['tx'])}")
37     print(f"Difficulty: {block['difficulty']}")
38
39     print("\n--- Blockchain Information ---")
40     binfo = proxy.getblockchaininfo()
41     print(f"Chain: {binfo['chain']}")
42     print(f"Blocks: {binfo['blocks']}")
43     print(f"Size on disk: {binfo['size_on_disk']} bytes")
44
45     print("\n--- Network Information ---")
46     ninfo = proxy.getnetworkinfo()
47     print(f"Version: {ninfo['version']}")
48     print(f"Subversion: {ninfo['subversion']}")
49     print(f"Connections: {ninfo['connections']}")
50
51     print("\n--- Mempool Information ---")

```

(continues on next page)

(continued from previous page)

```

52 minfo = proxy.getmempoolinfo()
53 print(f"Size: {minfo['size']} transactions")
54 print(f"Bytes: {minfo['bytes']} bytes")
55
56 print("\n--- Wallet Information ---")
57 try:
58     # These commands require a loaded wallet
59     balance = proxy.getbalance()
60     print(f"Balance: {balance} BTC")
61
62     new_addr = proxy.getnewaddress()
63     print(f"New address: {new_addr}")
64 except Exception as e:
65     print(f"Wallet commands failed (no wallet loaded?): {e}")
66
67
68 if __name__ == "__main__":
69     main()

```

2.12 Setup API

`bitcoinutils.setup.setup(network='testnet')`

Parameters

network (*str*)

Return type

str

`bitcoinutils.setup.get_network()`

Return type

str

`bitcoinutils.setup.set_security_warnings(enabled)`

Enable or disable warnings for pure-Python private-key operations.

Parameters

enabled (*bool*)

Return type

None

`bitcoinutils.setup.get_security_warnings()`

Return whether pure-Python private-key operation warnings are enabled.

Return type

bool

`bitcoinutils.setup.should_warn_about_private_key_use()`

Return True the first time a private-key operation should warn.

Return type

bool

`bitcoinutils.setup.is_mainnet()`

Return type

bool

`bitcoinutils.setup.is_testnet()`

Return type

bool

`bitcoinutils.setup.is_testnet4()`

Return type

bool

`bitcoinutils.setup.is_signet()`

Return type

bool

`bitcoinutils.setup.is_regtest()`

Return type

bool

2.13 Keys and Addresses API

class `bitcoinutils.keys.PrivateKey(wif=None, secret_exponent=None, b=None)`

Bases: `object`

Represents an ECDSA private key.

The object can be created randomly, from WIF, from raw 32-byte key material, or from a secret exponent. It can derive the corresponding `PublicKey` and sign Bitcoin messages and transaction digests.

Parameters

- **wif** (*Optional[str]*)
- **secret_exponent** (*Optional[int]*)
- **b** (*Optional[bytes]*)

key

The underlying ECDSA signing key object.

Common methods

```from_wif(wif)```

Create an object from WIF or WIFC text.

```from_bytes(b)```

Create an object from 32 raw private-key bytes.

```to_wif(compressed=True)```

Return the key as WIF or WIFC text.

```to_bytes()```

Return the raw 32-byte private key.

```
``get_public_key()``
```

Return the corresponding PublicKey.

```
``sign_message(message, compressed=True)``
```

Sign a Bitcoin message and return a compact Base64 signature.

```
``sign_input(tx, txin_index, script, sighash=SIGHASH_ALL)``
```

Sign a legacy transaction input.

```
``sign_segwit_input(tx, txin_index, script, amount, sighash=SIGHASH_ALL)``
```

Sign a SegWit v0 transaction input.

```
``sign_taproot_input(...)``
```

Sign a Taproot transaction input by key path or script path.

```
to_bytes()
```

Returns key's bytes

Return type

bytes

```
classmethod from_wif(wif)
```

Creates key from WIFC or WIF format key

Parameters

wif (*str*)

Return type

PrivateKey

```
classmethod from_bytes(b)
```

Creates a key directly from 32 raw bytes

Parameters

b (*bytes*)

Return type

PrivateKey

```
to_wif(compressed=True)
```

Returns key in WIFC or WIF string

Pseudocode:

```
network_prefix = (1 byte version number)
```

```
data = network_prefix + (32 bytes number/key) [ + 0x01 if compressed ]
```

```
data_hash = SHA-256( SHA-256( data ) )
```

```
checksum = (first 4 bytes of data_hash)
```

```
wif = Base58CheckEncode( data + checksum )
```

Parameters

compressed (*bool*)

Return type

str

sign_message(*message*, *compressed=True*)

Signs the message with the private key (deterministically)

Bitcoin uses a compact format for message signatures (for tx sigs it uses normal DER format). The format has the normal *r* and *s* parameters that ECDSA signatures have but also includes a prefix which encodes extra information. Using the prefix the public key can be reconstructed when verifying the signature.

Prefix values:

- 27 - 0x1B = first key with even *y*
- 28 - 0x1C = first key with odd *y*
- 29 - 0x1D = second key with even *y*
- 30 - 0x1E = second key with odd *y*

If key is compressed add 4 (31 - 0x1F, 32 - 0x20, 33 - 0x21, 34 - 0x22 respectively)

Returns a Bitcoin compact signature in Base64

Notes

This pure-Python signing path is not side-channel hardened.

Parameters

- **message** (*str*)
- **compressed** (*bool*)

Return type

str | *None*

sign_input(*tx*, *txin_index*, *script*, *sighash=1*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **script** (*Script*)
- **sighash** (*int*)

Return type

str

sign_segwit_input(*tx*, *txin_index*, *script*, *amount*, *sighash=1*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **script** (*Script*)
- **amount** (*int*)
- **sighash** (*int*)

Return type

str

sign_taproot_input(*tx*, *txin_index*, *utxo_scripts*, *amounts*, *script_path=False*, *tapleaf_script=[]*, *tapleaf_scripts=None*, *sighash=0*, *tweak=True*)

Parameters

- **tx** (*Transaction*)
- **txin_index** (*int*)
- **utxo_scripts** (*list[Script]*)
- **amounts** (*list[int]*)
- **script_path** (*bool*)
- **tapleaf_script** (*Script*)
- **tapleaf_scripts** (*Script | List[Script] | List[List[Script]] | bytes | None*)
- **sighash** (*int*)
- **tweak** (*bool*)

Return type

str

get_public_key()

Returns the corresponding *PublicKey*

Return type

PublicKey

class bitcoinutils.keys.**PublicKey**(*hex_str=None*, *message=None*, *signature=None*)

Bases: *object*

Represents an ECDSA public key.

Parameters

- **hex_str** (*Optional[str]*)
- **message** (*Optional[str]*)
- **signature** (*Optional[bytes]*)

key

the raw public key of 64 bytes (x, y coordinates of the ECDSA curve)

Type

bytes

from_hex(*hex_str*)

creates an object from a hex string in SEC format (classmethod)

Parameters

hex_str (*str*)

Return type

PublicKey

from_message_signature(*signature*)

NO-OP! (classmethod)

Parameters

- **message** (*str*)
- **signature** (*bytes*)

Return type
PublicKey

verify_message(*address, signature, message*) (*classmethod*)

constructs the public key, confirms the address and verifies the signature (classmethod)

Parameters

- **address** (*str*)
- **signature** (*str*)
- **message** (*str*)

Return type
bool

verify(*signature, message*)

returns true if the message was signed with this public key's corresponding private key.

Parameters

- **signature** (*str*)
- **message** (*str*)

Return type
bool

to_hex(*compressed=True*)

returns the key as hex string (in SEC format - compressed by default)

Parameters

compressed (*bool*)

Return type
str

to_x_only_hex(*script*)

returns the x coordinate only as hex string before tweaking (needed for taproot)

Return type
str

to_taproot_hex(*script*)

returns the x coordinate only as hex string after tweaking (needed for taproot)

Parameters

scripts (*Script | List[Script] | List[List[Script]] | None*)

Return type
Tuple[str, bool]

is_y_even()

returns true if y coordinate is even

Return type
bool

to_bytes()

returns the key's raw bytes

Return type

bytes

to_hash160()

returns the hash160 hex string of the public key

Parameters

compressed (*bool*)

Return type

str

get_address(compressed=True)

returns the corresponding P2pkhAddress object

Parameters

compressed (*bool*)

Return type

P2pkhAddress

get_segwit_address()

returns the corresponding P2wpkhAddress object

Return type

P2wpkhAddress

get_taproot_address(scripts)

returns the corresponding P2trAddress object

Parameters

scripts (*Script | List[Script] | List[List[Script]] | bytes | None*)

Return type

P2trAddress

classmethod from_hex(hex_str)

Creates a public key from a hex string (SEC format)

Parameters

hex_str (*str*)

Return type

PublicKey

to_bytes()

Returns key's bytes

Return type

bytes

to_hex(compressed=True)

Returns public key as a hex string (SEC format - compressed by default)

Parameters

compressed (*bool*)

Return type

str

to_x_only_hex()

Returns the x coordinate of the public key as hex string.

Return type

str

to_taproot_hex(*scripts=None*)

Returns the tweaked x coordinate of the public key as a hex string.

Parameters

scripts (*list[list[Script]]*) – a list of list of Scripts describing the merkle tree of scripts to commit

Return type

Tuple[str, bool]

is_y_even()

Returns True if the y coordinate of the public key is even and False otherwise.

Return type

bool

classmethod from_message_signature(*message, signature*)

Recovers a public key from a Bitcoin-signed message and a 65-byte compressed signature.

Parameters

- **message** (*str*)
- **signature** (*bytes*)

Return type

PublicKey

classmethod verify_message(*address, signature, message*)

Creates a public key from a message signature and verifies message

Bitcoin uses a compact format for message signatures (for tx sigs it uses normal DER format). The format has the normal r and s parameters that ECDSA signatures have but also includes a prefix which encodes extra information. Using the prefix the public key can be reconstructed from the signature.

Prefix values:

- 27 - 0x1B = first key with even y
- 28 - 0x1C = first key with odd y
- 29 - 0x1D = second key with even y
- 30 - 0x1E = second key with odd y

If key is compressed add 4 (31 - 0x1F, 32 - 0x20, 33 - 0x21, 34 - 0x22 respectively)

Raises

ValueError – If signature is invalid

Parameters

- **address** (*str*)
- **signature** (*str*)
- **message** (*str*)

Return type

bool

verify(*signature*, *message*)

Verifies that the message was signed with this public key's corresponding private key.

Parameters

- **signature** (*str*)
- **message** (*str*)

Return type

bool

to_hash160(*compressed=True*)

Returns the RIPEMD(SHA256()) of the public key in hex

Parameters**compressed** (*bool*)**Return type**

str

get_address(*compressed=True*)

Returns the corresponding P2PKH Address (default compressed)

Parameters**compressed** (*bool*)**Return type***P2pkhAddress***get_segwit_address**()

Returns the corresponding P2WPKH address

Only compressed is allowed. It is otherwise identical to normal P2PKH address.

Return type*P2wpkhAddress***get_taproot_address**(*scripts=None*)

Returns the corresponding P2TR address

Only compressed is allowed. Taproot uses x-only public key with even y (02 compressed keys). By default tagged_hashes are used.

scripts contains the list of lists of Scripts describing the merkle tree

Parameters**scripts** (*Script | List[Script] | List[List[Script]] | bytes | None*)**Return type***P2trAddress***class** bitcoinutils.keys.**Address**(*address=None*, *hash160=None*, *script=None*)

Bases: ABC

Represents a Bitcoin address

hash160

the hash160 string representation of the address; hash160 represents two consecutive hashes of the public key or the redeem script, first a SHA-256 and then an RIPEMD-160

Type*str***from_address**(*address*)

instantiates an object from address string encoding

Parameters**address** (*str*)**Return type***Address***from_hash160**(*hash160_str*)

instantiates an object from a hash160 hex string

Parameters**hash160** (*str*)**Return type***Address***from_script**(*redeem_script*)

instantiates an object from a redeem_script

Parameters**script** (*Script*)**Return type***Address***to_string**()

returns the address's string encoding

Return type*str***to_hash160**()

returns the address's hash160 hex string representation

Return type*str***Raises**

- **TypeError** – No parameters passed
- **ValueError** – If an invalid address or hash160 is provided.

Parameters

- **address** (*Optional[str]*)
- **hash160** (*Optional[str]*)
- **script** (*Optional[Script]*)

classmethod from_address(*address*)

Creates an address object from an address string

Parameters**address** (*str*)**Return type***Address*

classmethod `from_hash160(hash160)`

Creates an address object from a hash160 string

Parameters

hash160 (*str*)

Return type

Address

classmethod `from_script(script)`

Creates an address object from a Script object

Parameters

script (*Script*)

Return type

Address

to_hash160()

Returns as hash160 hex string

Return type

str

get_type()

Returns the type of address

Return type

str

to_string()

Returns as address string

Pseudocode:

network_prefix = (1 byte version number)

data = network_prefix + hash160_bytes

data_hash = SHA-256(SHA-256(hash160_bytes))

checksum = (first 4 bytes of data_hash)

address_bytes = Base58CheckEncode(data + checksum)

Return type

str

to_script_pub_key()

Overriden from subclasses

Return type

Script

class `bitcoinutils.keys.P2pkhAddress(address=None, hash160=None)`

Bases: *Address*

Encapsulates a P2PKH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)

- **hash160** (*Optional[str]*)

to_script_pub_key()

returns the scriptPubKey (P2PKH) that corresponds to this address

Return type

Script

get_type()

returns the type of address

Return type

str

to_script_pub_key()

Returns the scriptPubKey (P2PKH) that corresponds to this address

Return type

Script

get_type()

Returns the type of address

Return type

str

class bitcoinutils.keys.**P2shAddress**(*address=None, hash160=None, script=None*)

Bases: Address

Encapsulates a P2SH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **hash160** (*Optional[str]*)
- **script** (*Optional[Script]*)

to_script_pub_key()

returns the scriptPubKey (P2SH) that corresponds to this address

Return type

Script

get_type()

returns the type of address

Return type

str

to_script_pub_key()

Returns the scriptPubKey (P2SH) that corresponds to this address

Return type

Script

get_type()

Returns the type of address

Return type

str

class bitcoinutils.keys.**SegwitAddress**(*address=None, witness_program=None, script=None, version='p2wpkhv0'*)

Bases: ABC

Represents a Bitcoin segwit address

Note that currently the python bech32[m] reference implementation is used (by Pieter Wuille).

witness_program

for segwit v0 this is the hash string representation of either the address; it can be either a public key hash (P2WPKH) or the hash of the script (P2WSH)

for segwit v1 (aka taproot) this is the public key

Type

str

from_address(*address*)

instantiates an object from address string encoding

Parameters

address (*str*)

Return type

SegwitAddress

from_program(*hash_str*)

instantiates an object from a witness program hex string

from_script(*witness_script*)

instantiates an object from a witness_script

Parameters

script (*Script*)

Return type

SegwitAddress

to_string()

returns the address's string encoding (Bech32)

Return type

str

to_hash()

returns the address's hash hex string representation

Raises

- **TypeError** – No parameters passed
- **ValueError** – If an invalid address or hash is provided.

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **script** (*Optional[Script]*)

- **version** (*str*)

classmethod from_address(*address*)

Creates an address object from an address string

Parameters

address (*str*)

Return type

SegwitAddress

classmethod from_witness_program(*witness_program*)

Creates an address object from a hash string

Parameters

witness_program (*str*)

Return type

SegwitAddress

classmethod from_script(*script*)

Creates an address object from a Script object

Parameters

script (*Script*)

Return type

SegwitAddress

to_witness_program()

Returns witness program as hex string

Return type

str

to_string()

Returns as address string

Uses a segwit's python reference implementation for now. (TODO)

Return type

str

to_script_pub_key()

Overriden from subclasses

Return type

Script

class bitcoinutils.keys.**P2wpkhAddress**(*address=None, witness_program=None, version='p2wpkhv0'*)

Bases: *SegwitAddress*

Encapsulates a P2WPKH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **version** (*str*)

to_script_pub_key()

returns the scriptPubKey of a P2WPKH witness script

Return type

Script

get_type()

returns the type of address

Return type

str

to_script_pub_key()

Returns the scriptPubKey of a P2WPKH witness script

Return type

Script

get_type()

Returns the type of address

Return type

str

class bitcoinutils.keys.**P2wshAddress**(*address=None, witness_program=None, script=None, version='p2wshv0'*)

Bases: SegwitAddress

Encapsulates a P2WSH address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **script** (*Optional[Script]*)
- **version** (*str*)

from_script(witness_script)

instantiates an object from a witness_script

get_type()

returns the type of address

Return type

str

to_script_pub_key()

Returns the scriptPubKey of a P2WPKH witness script

Return type

Script

get_type()

Returns the type of address

Return type

str

```
class bitcoinutils.keys.P2trAddress(address=None, witness_program=None, version='p2trv1',
                                     is_odd=False)
```

Bases: SegwitAddress

Encapsulates a P2TR (Taproot) address.

Check Address class for details

Parameters

- **address** (*Optional[str]*)
- **witness_program** (*Optional[str]*)
- **version** (*str*)
- **is_odd** (*bool*)

to_script_pub_key()

returns the scriptPubKey of a P2TR witness script

Return type

Script

get_type()

returns the type of address

Return type

str

to_script_pub_key()

Returns the scriptPubKey of a P2TR witness script

Return type

Script

get_type()

Returns the type of address

Return type

str

is_odd()

Returns True if the y coordinate of the public key is odd and False otherwise.

Return type

bool

2.14 Script API

```
class bitcoinutils.script.Script(script)
```

Represents any script in Bitcoin

A Script contains just a list of OP_CODES and also knows how to serialize into bytes

script

the list with all the script OP_CODES and data

Type

list

to_bytes()

returns a serialized byte version of the script

Return type

bytes

to_hex()

returns a serialized version of the script in hex

Return type

str

get_script()

returns the list of strings that makes up this script

Return type

list[Any]

copy()

creates a copy of the object (classmethod)

Parameters

script (*Script*)

Return type

Script

from_raw()**Parameters**

- **scriptrawhex** (*str* / *bytes*)
- **has_segwit** (*bool*)

to_p2sh_script_pub_key()

converts script to p2sh scriptPubKey (locking script)

Return type

Script

to_p2wsh_script_pub_key()

converts script to p2wsh scriptPubKey (locking script)

Return type

Script

Raises

ValueError – If string data is too large or integer is negative

Parameters

script (*list[Any]*)

classmethod copy(*script*)

Deep copy of Script

Parameters

script (*Script*)

Return type

Script

to_bytes()

Converts the script to bytes

If an OP code the appropriate byte is included according to: <https://en.bitcoin.it/wiki/Script> If not consider it data (signature, public key, public key hash, etc.) and include with appropriate OP_PUSHDATA OP code plus length

Return type

bytes

to_hex()

Converts the script to hexadecimal

Return type

str

static from_raw(*scriptrawhex*, *has_segwit=False*)

Import a Script commands list from raw hexadecimal data.

Parameters

- **scriptrawhex** (*str* or *bytes*) – The raw script as hexadecimal text or bytes.
- **has_segwit** (*bool*) – Whether to parse using SegWit-specific script handling.

get_script()

Returns script as array of strings

Return type

list[*Any*]

to_p2sh_script_pub_key()

Converts script to p2sh scriptPubKey (locking script)

Calculates hash160 of the script and uses it to construct a P2SH script.

Return type

Script

to_p2wsh_script_pub_key()

Converts script to p2wsh scriptPubKey (locking script)

Calculates the sha256 of the script and uses it to construct a P2WSH script.

Return type

Script

bitcoinutils.script.is_p2pkh(*script*)

Returns True for a P2PKH scriptPubKey.

Parameters

script (*Script*)

Return type

bool

bitcoinutils.script.is_p2sh(*script*)

Returns True for OP_HASH160 <20-byte-hash> OP_EQUAL.

Parameters

script (*Script*)

Return type

bool

`bitcoinutils.script.is_p2wpkh(script)`

Returns True for OP_0 <20-byte-hash>.

Parameters**script** (*Script*)**Return type**

bool

`bitcoinutils.script.is_p2wsh(script)`

Returns True for OP_0 <32-byte-hash>.

Parameters**script** (*Script*)**Return type**

bool

`bitcoinutils.script.is_p2tr(script)`

Returns True for OP_1 <32-byte-key>.

Parameters**script** (*Script*)**Return type**

bool

2.15 Transactions API

class `bitcoinutils.transactions.TxInput`(*txid*, *txout_index*, *script_sig=None*, *sequence=b'\xfd\xff\xff\xff'*)

Bases: object

Represents a transaction input.

A transaction input requires a transaction id of a UTXO and the index of that UTXO.

Parameters

- **txid** (*str*)
- **txout_index** (*int*)
- **script_sig** (*Script* | *None*)
- **sequence** (*str* | *bytes*)

txid

the transaction id as a hex string (little-endian as displayed by tools)

Type

str

txout_index

the index of the UTXO that we want to spend

Type

int

script_sig

the script that satisfies the locking conditions (aka unlocking script)

Type

list (strings)

sequence

the input sequence (for timelocks, RBF, etc.)

Type

bytes

to_bytes()

serializes TxInput to bytes

Return type

bytes

copy()

creates a copy of the object (classmethod)

Parameters

txin (*TxInput*)

Return type

TxInput

from_raw()

instantiates object from raw hex input (classmethod)

Parameters

- **txinputrawhex** (*str* | *bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

to_bytes()

Serializes to bytes

Return type

bytes

static from_raw(txinputrawhex, cursor=0, has_segwit=False)

Imports a TxInput from a Transaction's hexadecimal data

Parameters

- **txinputrawhex** (*str* | *bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

txinputrawhex

The hexadecimal raw string of the Transaction

Type

string (hex)

cursor

The cursor of which the algorithm will start to read the data

Type

int

has_segwit

Is the Tx Input segwit or not

Type

boolean

classmethod copy(*txin*)

Deep copy of TxInput

Parameters

txin (*TxInput*)

Return type

TxInput

class bitcoinutils.transactions.**TxWitnessInput**(*stack*)

Bases: object

A list of the witness items required to satisfy the locking conditions of a segwit input (aka witness stack).

Parameters

stack (*list[str]*)

stack

the witness items (hex str) list

Type

list

to_bytes()

returns a serialized byte version of the witness items list

Return type

bytes

copy()

creates a copy of the object (classmethod)

Parameters

txwin (*TxWitnessInput*)

Return type

TxWitnessInput

to_bytes()

Converts to bytes

Return type

bytes

classmethod copy(*txwin*)

Deep copy of TxWitnessInput

Parameters

txwin (*TxWitnessInput*)

Return type*TxWitnessInput***class** bitcoinutils.transactions.**TxOutput**(*amount, script_pubkey*)

Bases: object

Represents a transaction output

Parameters

- **amount** (*int*)
- **script_pubkey** (*Script*)

amount

the value we want to send to this output in satoshis

Type*int***script_pubkey**

the script that will lock this amount

Type*Script***to_bytes()**

serializes TxInput to bytes

Return type*bytes***copy()**

creates a copy of the object (classmethod)

Parameters

- **txout** (*TxOutput*)

Return type*TxOutput***from_raw()**

instantiates object from raw hex output (classmethod)

Parameters

- **txoutputrawhex** (*str | bytes*)
- **cursor** (*int*)
- **has_segwit** (*bool*)

to_bytes()

Serializes to bytes

Return type*bytes***static from_raw**(*txoutputrawhex, cursor=0, has_segwit=False*)

Imports a TxOutput from a Transaction's hexadecimal data

Parameters

- **txoutputrawhex** (*str | bytes*)

- **cursor** (*int*)
- **has_segwit** (*bool*)

txoutputrawhex

The hexadecimal raw string of the Transaction

Type

string (hex)

cursor

The cursor of which the algorithm will start to read the data

Type

int

has_segwit

Is the Tx Output segwit or not

Type

boolean

classmethod copy(txout)

Deep copy of TxOutput

Parameters

txout (*TxOutput*)

Return type

TxOutput

class bitcoinutils.transactions.**Sequence**(*seq_type, value, is_type_block=True*)

Bases: object

Helps setting up appropriate sequence. Used to provide the sequence to transaction inputs and to scripts.

value

The value of the block height or the 512 seconds increments

Type

int

seq_type

Specifies the type of sequence (TYPE_RELATIVE_TIMELOCK | TYPE_ABSOLUTE_TIMELOCK | TYPE_REPLACE_BY_FEE)

Type

int

is_type_block

If type is TYPE_RELATIVE_TIMELOCK then this specifies its type (block height or 512 secs increments)

Type

bool

for_input_sequence()

Serializes the relative sequence as required in a transaction

Return type

str | bytes | None

for_script()

Returns the appropriate integer for a script; e.g. for relative timelocks

Return type

int

Raises

ValueError – if the value is not within range of 2 bytes.

Parameters

- **seq_type** (*int*)
- **value** (*int*)
- **is_type_block** (*bool*)

for_input_sequence()

Creates a relative timelock sequence value as expected from TxInput sequence attribute

Return type

str | bytes | None

for_script()

Creates a relative/absolute timelock sequence value as expected in scripts

Return type

int

class bitcoinutils.transactions.**Locktime**(*value*)

Bases: object

Helps setting up appropriate locktime.

value

The value of the block height or the Unix epoch (seconds from 1 Jan 1970 UTC)

Type

int

for_transaction()

Serializes the locktime as required in a transaction

Return type

bytes

Raises

ValueError – if the value is not within range of 2 bytes.

Parameters

value (*int*)

for_transaction()

Creates a timelock as expected from Transaction

Return type

bytes

```
class bitcoinutils.transactions.Transaction(inputs=None, outputs=None,  
                                             locktime=b'\x00\x00\x00\x00',  
                                             version=b'\x02\x00\x00\x00', has_segwit=False,  
                                             witnesses=None)
```

Bases: object

Represents a Bitcoin transaction.

A transaction contains inputs, outputs, version, locktime, and optionally SegWit witness stacks. It can serialize itself, parse raw transactions, compute txids/wtxids, and produce legacy, SegWit v0, and Taproot signing digests.

Parameters

- **inputs** (*list[TxInput] | None*)
- **outputs** (*list[TxOutput] | None*)
- **locktime** (*str | bytes*)
- **version** (*bytes*)
- **has_segwit** (*bool*)
- **witnesses** (*list[TxWitnessInput] | None*)

inputs

Transaction inputs.

Type

list[TxInput]

outputs

Transaction outputs.

Type

list[TxOutput]

locktime

Transaction locktime.

Type

bytes

version

Transaction version.

Type

bytes

has_segwit

Whether the transaction should serialize with SegWit marker/flag.

Type

bool

witnesses

Witness stacks corresponding to inputs.

Type

list[TxWitnessInput]

Common methods

```
``from_raw(rawtxhex)``
```

Instantiate a transaction from serialized raw hexadecimal data.

```
``to_bytes(has_segwit)``
```

Serialize the transaction to bytes.

```
``to_hex()`` / ``serialize()``
```

Serialize the transaction to hexadecimal text.

```
``get_txid()``
```

Return the transaction id without witness data.

```
``get_wtxid()``
```

Return the witness transaction id.

```
``get_size()`` / ``get_vsize()``
```

Return byte size and virtual size.

```
``copy(tx)``
```

Deep-copy a transaction.

```
``set_witness(txin_index, witness)``
```

Set the witness stack for an input.

```
``get_transaction_digest(...)``
```

Return the legacy signing digest.

```
``get_transaction_segwit_digest(...)``
```

Return the SegWit v0 signing digest.

```
``get_transaction_taproot_digest(...)``
```

Return the Taproot signing digest.

```
static from_raw(rawtxhex)
```

Imports a Transaction from hexadecimal data.

Parameters

rawtxhex (*str* / *bytes*)

rawtxhex

The hexadecimal raw string of the Transaction.

Type

string (hex)

```
classmethod copy(tx)
```

Deep copy of Transaction

Parameters

tx (*Transaction*)

Return type

Transaction

```
set_witness(txin_index, witness)
```

Safely set a witness at the specified index

Parameters

- **txin_index** (*int*)

- **witness** (*TxWitnessInput*)

get_transaction_digest(*txin_index*, *script*, *sighash=1*)

Returns the transaction's digest for signing. https://en.bitcoin.it/wiki/OP_CHECKSIG

SIGHASH types (see constants.py):

- SIGHASH_ALL - signs all inputs and outputs (default)
- SIGHASH_NONE - signs all of the inputs
- SIGHASH_SINGLE - signs all inputs but only *txin_index* output
- SIGHASH_ANYONECANPAY (only combined with one of the above)
 - with ALL - signs all outputs but only *txin_index* input
 - with NONE - signs only the *txin_index* input
 - with SINGLE - signs *txin_index* input and output

Parameters

- **txin_index** (*int*)
- **script** (*Script*)
- **sighash** (*int*)

txin_index

The index of the input that we wish to sign

Type
int

script

The scriptPubKey of the UTXO that we want to spend

Type
list (string)

sighash

The type of the signature hash to be created

Type
int

get_transaction_segwit_digest(*txin_index*, *script*, *amount*, *sighash=1*)

Returns the segwit v0 transaction's digest for signing. <https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki>

SIGHASH types (see constants.py):

- SIGHASH_ALL - signs all inputs and outputs (default)
- SIGHASH_NONE - signs all of the inputs
- SIGHASH_SINGLE - signs all inputs but only *txin_index* output
- SIGHASH_ANYONECANPAY (only combined with one of the above)
 - with ALL - signs all outputs but only *txin_index* input
 - with NONE - signs only the *txin_index* input
 - with SINGLE - signs *txin_index* input and output

Parameters

- **txin_index** (*int*) – The index of the input that we wish to sign.
- **script** (*Script*) – The scriptCode that corresponds to the SegWit output being spent.

- **amount** (*int*) – The amount of the UTXO being spent, in satoshis.
- **sighash** (*int*) – The signature hash type to create.

get_transaction_taproot_digest(*txin_index*, *script_pubkeys*, *amounts*, *ext_flag=0*, *script=None*, *leaf_ver=192*, *sighash=0*)

Returns the segwit v1 (taproot) transaction's digest for signing. <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki> Also consult Bitcoin Core code at: <https://github.com/bitcoin/bitcoin/blob/29c36f070618ea5148cd4b2da3732ee4d37af66b/src/script/interpreter.cpp#L1478> And: https://github.com/bitcoin/bitcoin/blob/b5f33ac1f82aea290b4653af36ac2ad1bf1cce7b/test/functional/test_framework/script.py

SIGHASH types (see constants.py):

- TAPROOT_SIGHASH_ALL - signs all inputs and outputs (default)
- SIGHASH_ALL - signs all inputs and outputs
- SIGHASH_NONE - signs all of the inputs
- SIGHASH_SINGLE - signs all inputs but only txin_index output
- SIGHASH_ANYONECANPAY (only combined with one of the above)
- with ALL - signs all outputs but only txin_index input
- with NONE - signs only the txin_index input
- with SINGLE - signs txin_index input and output

Parameters

- **txin_index** (*int*) – The index of the input that we wish to sign.
- **script_pubkeys** (*list[Script]*) – The scriptPubKeys that correspond to all inputs/UTXOs.
- **amounts** (*list[int]*) – The amounts that correspond to all inputs/UTXOs, in satoshis.
- **ext_flag** (*int*) – Extension mechanism. Use 0 for key path and 1 for script path.
- **script** (*Script*, *optional*) – The tapleaf script being spent when ext_flag is 1.
- **leaf_ver** (*int*) – The script version. Defaults to LEAF_VERSION_TAPSCRIPT.
- **sighash** (*int*) – The signature hash type to create.

to_bytes(*has_segwit*)

Serializes to bytes

Parameters

has_segwit (*bool*)

Return type

bytes

get_txid()

Hashes the serialized (bytes) tx to get a unique id

Return type

str

get_wtxid()

Hashes the serialized (bytes) tx including segwit marker and witnesses

Return type

str

get_size()

Gets the size of the transaction

Return type

int

get_vsize()

Gets the virtual size of the transaction.

For non-segwit txs this is identical to `get_size()`. For segwit txs the marker and witnesses length needs to be reduced to 1/4 of its original length. Thus it is subtracted from size and then it is divided by 4 before added back to size to produce vsize (always rounded up).

https://en.bitcoin.it/wiki/Weight_units

Return type

int

to_hex()

Converts object to hexadecimal string

Return type

str

serialize()

Converts object to hexadecimal string

Return type

str

to_psbt()

Create a PSBT from this transaction.

Returns

A new PSBT wrapping this transaction.

Return type

PSBT

classmethod from_psbt(*psbt_b64_or_hex*)

Extract a signed transaction from a finalized PSBT.

Parameters

psbt_b64_or_hex (*str*) – Base64- or hex-encoded PSBT string.

Returns

The extracted signed transaction.

Return type

Transaction

2.16 HD Wallet API

Minimal BIP-32/BIP-39 HD wallet support.

This module intentionally implements only the functionality used by bitcoinutils: deriving private keys from a mnemonic or an extended private key and returning them as `bitcoinutils.keys.PrivateKey` objects.

class bitcoinutils.hdwallet.**HDWallet**(*xprivate_key=None, path=None, mnemonic=None*)

Minimal HD wallet wrapper used by bitcoinutils examples and tests.

The wrapper supports deriving Bitcoin private keys from a BIP-39 mnemonic or an extended private key. It does not implement the full external hdwallet package API.

Parameters

- **xprivate_key** (*Optional[str]*)
- **path** (*Optional[str]*)
- **mnemonic** (*Optional[str]*)

classmethod **from_mnemonic**(*mnemonic*)

Instantiate from a BIP-39 mnemonic code.

Parameters

mnemonic (*str*)

classmethod **from_xprivate_key**(*xprivate_key, path=None*)

Instantiate from an extended private key and derivation path.

Parameters

- **xprivate_key** (*str*)
- **path** (*str | None*)

from_path(*path*)

Derive and select a private key from an absolute BIP-32 path.

Parameters

path (*str*)

get_private_key()

Return a PrivateKey object used throughout bitcoinutils.

2.17 PSBT API

BIP-0174 Partially Signed Bitcoin Transaction (PSBT) support.

Implements PSBT version 0 per <https://github.com/bitcoin/bips/blob/master/bip-0174.mediawiki>

class bitcoinutils.psbt.**PSBTInput**

Bases: object

Data associated with a single PSBT input.

class bitcoinutils.psbt.**PSBTOutput**

Bases: object

Data associated with a single PSBT output.

class bitcoinutils.psbt.**PSBT**(*tx*)

Bases: object

BIP-174 Partially Signed Bitcoin Transaction (version 0).

Parameters

tx (*Transaction*)

tx

The unsigned transaction (no scriptSigs, no segwit flag).

Type

Transaction

inputs

Per-input metadata, partial signatures, etc.

Type

list[PSBTInput]

outputs

Per-output metadata.

Type

list[PSBTOutput]

unknown_global

Unknown global key-value pairs.

Type

dict[bytes, bytes]

to_bytes()

Serialize this PSBT to binary format.

Return type

bytes

to_base64()

Serialize to base64 string.

Return type

str

to_hex()

Serialize to hex string.

Return type

str

classmethod from_bytes(*data*)

Deserialize a PSBT from raw bytes.

Parameters

data (*bytes*)

Return type

PSBT

classmethod from_base64(*b64_str*)

Deserialize a PSBT from a base64 string.

Parameters

b64_str (*str*)

Return type

PSBT

classmethod `from_hex(hex_str)`

Deserialize a PSBT from a hex string.

Parameters

hex_str (*str*)

Return type

PSBT

update_input(*index*, *non_witness_utxo*=None, *witness_utxo*=None, *redeem_script*=None, *witness_script*=None, *sighash_type*=None, *bip32_derivs*=None)

Add UTXO and script metadata for an input.

Parameters

- **index** (*int*)
- **non_witness_utxo** (*Transaction* | None)
- **witness_utxo** (*TxOutput* | None)
- **redeem_script** (*Script* | None)
- **witness_script** (*Script* | None)
- **sighash_type** (*int* | None)
- **bip32_derivs** (*dict*[bytes, *tuple*[bytes, *list*[int]]] | None)

update_output(*index*, *redeem_script*=None, *witness_script*=None, *bip32_derivs*=None)

Add script metadata for an output.

Parameters

- **index** (*int*)
- **redeem_script** (*Script* | None)
- **witness_script** (*Script* | None)
- **bip32_derivs** (*dict*[bytes, *tuple*[bytes, *list*[int]]] | None)

sign_input(*input_index*, *private_key*, *sighash*=1)

Sign a PSBT input using the library's signing methods.

Parameters

- **input_index** (*int*) – Index of the input to sign.
- **private_key** (*PrivateKey*) – The private key to sign with.
- **sighash** (*int*) – Sighash type (default SIGHASH_ALL).

Returns

True if a signature was produced.

Return type

bool

combine(*other*)

Combine this PSBT with another, returning a new merged PSBT.

Both PSBTs must have the same unsigned transaction.

Parameters

other (*PSBT*)

Return type*PSBT***finalize_input**(*input_index*)

Finalize a single input, constructing final scriptSig / witness.

Parameters**input_index** (*int*)**finalize**()

Finalize all inputs.

extract_transaction()

Extract the fully signed transaction.

Returns

The complete, signed transaction ready for broadcast.

Return type

Transaction

2.18 Block API

```
class bitcoinutils.block.BlockHeader(version=None, previous_block_hash=None, merkle_root=None,  
timestamp=None, target_bits=None, nonce=None)
```

Bases: object

Represents a Bitcoin block header. This class encapsulates the details of a block's header in the blockchain, which includes essential mining and blockchain continuity information.

Parameters

- **version** (*int* / *None*)
- **previous_block_hash** (*bytes* / *None*)
- **merkle_root** (*bytes* / *None*)
- **timestamp** (*int* / *None*)
- **target_bits** (*int* / *None*)
- **nonce** (*int* / *None*)

version

Version of the block protocol.

Type

Optional[int]

previous_block_hash

256-bit hash of the previous block.

Type

Optional[bytes]

merkle_root

256-bit hash based on all of the transactions in the block.

Type

Optional[bytes]

timestamp

Block creation time in seconds since the Unix epoch.

Type

Optional[int]

target_bits

Encoded difficulty target as a compact format.

Type

Optional[int]

nonce

Arbitrary number that is adjusted to achieve the required proof of work.

Type

Optional[int]

__init__(*version, previous_block_hash, merkle_root, timestamp, target_bits, nonce*)

Initializes a new instance of the BlockHeader.

from_raw(*rawhexdata*)

Constructs a BlockHeader object from raw block header data.

Parameters

rawhexdata (*str* / *bytes*)

__str__()

Provides a human-readable representation of the BlockHeader.

__repr__()

Provides a precise string representation of the BlockHeader object that could be used to recreate the object.

get_version()

Retrieves the version of the block.

Return type

int | None

get_previous_block_hash()

Retrieves the hash of the previous block.

Return type

bytes | None

get_merkle_root()

Retrieves the Merkle root of the transactions in the block.

Return type

bytes | None

get_timestamp()

Retrieves the timestamp of when the block was created.

Return type

int | None

get_target_bits()

Retrieves the target bits that encode the mining difficulty target.

Return type

int | None

get_nonce()

Retrieves the nonce used for mining to achieve the block's proof of work.

Return type

int | None

format_timestamp()

Formats the timestamp into a human-readable UTC datetime string.

get_target_hex()

Decodes target bits into the full proof-of-work target.

static from_raw(*rawhexdata*)

Constructs a BlockHeader object from a raw block header data.

Parameters

rawhexdata (*Union[str, bytes]*) – Raw hexadecimal or byte data representing the block header.

Returns

An instance of BlockHeader initialized from the provided raw data.

Return type

BlockHeader

Raises

- **TypeError** – If the input data type is not a string or bytes.
- **ValueError** – If the length of raw data does not match the expected size of a block header.

get_version()

Returns the block version, or None if not set.

Return type

int | None

get_previous_block_hash()

Returns the previous block hash as bytes, or None if not set.

Return type

bytes | None

get_merkle_root()

Returns the merkle root as bytes, or None if not set.

Return type

bytes | None

get_timestamp()

Returns the block timestamp, or None if not set.

Return type

int | None

get_target_bits()

Returns the compact form of the target difficulty (*target_bits*), or None if not set.

Return type

int | None

get_nonce()

Returns the nonce used for mining, or None if not set.

Return type
int | None

format_timestamp()

Formats the block's timestamp into a human-readable UTC datetime string.

Returns
The formatted UTC datetime string representing the block's timestamp.

Return type
str

get_target_hex()

Decodes the compact representation of the target bits into the full target hash that a block's hash must be less than or equal to, in order to solve the block.

Returns
The target value as a 64-character hexadecimal string, representing
the full 256-bit target hash used in the proof of work.

Return type
str

serialize_header()

Serializes the block header in the format required for hashing.

get_block_hash()

Calculates the block hash by double SHA-256 hashing the header.

class bitcoinutils.block.**Block**(*magic=None, block_size=None, header=None, transaction_count=None, transactions=None*)

Bases: object

Represents a Bitcoin block, encapsulating the block's fundamental components including the block header and a list of transactions.

Parameters

- **magic** (*bytes* | *None*)
- **block_size** (*int* | *None*)
- **header** (*BlockHeader* | *None*)
- **transaction_count** (*int* | *None*)
- **transactions** (*list[Transaction]* | *None*)

magic

Magic value indicating the network for which the block was intended.

Type
Optional[bytes]

block_size

Size of the block in bytes.

Type
Optional[int]

header

Header information of the block including version, previous block hash, etc.

Type

Optional[BlockHeader]

transaction_count

Number of transactions included in the block.

Type

Optional[int]

transactions

A list of transactions contained in the block.

Type

Optional[list[Transaction]]

__init__(*magic, block_size, header, transaction_count, transactions*)

Initializes a new instance of the Block class with specified attributes.

from_raw(*rawhexdata*)

Constructs a Block object from a raw block data. Raises TypeError if the input is neither bytes nor a hexadecimal string. Raises ValueError if the length of raw data does not match the expected size.

Parameters

rawhexdata (*str* / *bytes*)

__str__()

Provides a human-readable representation of the Block, showing all attributes.

__repr__()

Provides a precise string representation of the Block object that could be used to recreate the object.

get_magic_bytes()

Retrieves the block's magic bytes as a tuple containing the hexadecimal representation and a descriptive string. Raises ValueError if the magic bytes are not set.

Return type

tuple[str, str]

get_block_size()

Retrieves the size of the block in bytes.

Return type

int

get_block_header()

Retrieves the BlockHeader object associated with the block. Raises ValueError if the block header is not set.

Return type

BlockHeader

get_transactions()

Retrieves the list of Transaction objects contained in the block. Raises ValueError if there are no transactions.

Return type

list[Transaction]

get_transactions_count()

Retrieves the count of transactions contained in the block. Raises `ValueError` if there are no transactions.

Return type

`int`

get_coinbase_transaction()

Retrieves the first transaction in the block, typically the coinbase transaction. Raises `ValueError` if there are no transactions.

Return type

Transaction

get_block_reward()

Calculates the total output amount of the coinbase transaction, representing the block reward.

Return type

`int`

get_witness_transactions()

Returns a list of transactions that include SegWit data.

Return type

`list[Transaction]`

get_legacy_transactions()

Returns a list of transactions that are non-SegWit, legacy-style transactions.

Return type

`list[Transaction]`

static from_raw(rawhexdata)

Constructs a `Block` instance from raw block data in hexadecimal or byte format.

Parameters

rawhexdata (*Union[str, bytes]*) – The raw data of the block in hexadecimal or bytes format.

Returns

A fully parsed `Block` object.

Return type

`Block`

Raises

- **TypeError** – If the input is not a string or bytes.
- **ValueError** – If the input does not meet the expected block structure or size.

get_magic_bytes()

Retrieves the magic bytes and its corresponding network description.

Returns

The hexadecimal representation of the magic bytes and its description.

Return type

`tuple[str, str]`

Raises

ValueError – If the magic bytes are not set.

get_block_size()

Retrieves the size of the block in bytes.

Returns

The size of the block.

Return type

int

get_block_header()

Retrieves the header of the block.

Returns

The block header object.

Return type

BlockHeader

Raises

ValueError – If the block header is not set.

get_transactions()

Retrieves the transactions contained in the block.

Returns

A list of transactions.

Return type

list[Transaction]

Raises

ValueError – If there are no transactions in the block.

get_transactions_count()

Retrieves the number of transactions in the block.

Returns

The number of transactions.

Return type

int

Raises

ValueError – If there are no transactions.

get_coinbase_transaction()

Retrieves the coinbase transaction from the block, which is the first transaction in the block and contains the block reward.

Returns

The coinbase transaction.

Return type

Transaction

Raises

ValueError – If there are no transactions in the block.

get_block_reward()

Calculates the total amount of the block reward by summing the outputs of the coinbase transaction.

Returns

The total output amount in the coinbase transaction, representing the block reward.

Return type

int

Raises**ValueError** – If there are no transactions in the block.**get_witness_transactions()**

Return transactions that contain SegWit data.

Returns

Transactions with SegWit data.

Return type

list[Transaction]

get_legacy_transactions()

Return legacy transactions.

Returns

Transactions without SegWit data.

Return type

list[Transaction]

2.19 Utilities API

class bitcoinutils.utils.Secp256k1Params**class** bitcoinutils.utils.ControlBlock(*pubkey, scripts, index, is_odd=False*)

Represents a control block for spending a taproot script path

Parameters

- **pubkey** (*PublicKey*)
- **scripts** (*None | list[list[Script]]*)
- **index** (*int*)

pubkey

the internal public key object

Type

PublicKey

scripts

a list of list of Scripts describing the merkle tree of scripts to commit

Type

list[list[Script]]

merkle_path

the pre-calculated merkle path

Type

bytes

to_bytes()

returns the control block as bytes

Return type

bytes

to_hex()

returns the control block as a hexadecimal string

Return type

str

to_bytes()**Return type**

bytes

to_hex()

Converts object to hexadecimal string

Returns

The control block as a hexadecimal string

Return type

str

bitcoinutils.utils.get_tag_hashed_merkle_root(*scripts*)

Tag hashed merkle root of all scripts - tag hashes tapleafs and branches as needed.

Scripts is a list of list of Scripts describing the merkle tree of scripts to commit Example of scripts' list: [[A, B], C]

Parameters**scripts** (*None* | *Script* | *list[Script]* | *list[list[Script]]*)**Return type**

bytes

bitcoinutils.utils.to_satoshis(*num*)

Converts from any number type (int/float/Decimal) to satoshis (int)

Parameters**num** (*int* | *float* | *Decimal*)**bitcoinutils.utils.prepend_compact_size(*data*)**

Counts bytes and returns them with their varint (or compact size) prepended.

Parameters**data** (*bytes*)**Return type**

bytes

bitcoinutils.utils.encode_varint(*i*)

Encode a potentially very large integer into varint bytes. The length should be specified in little-endian.

<https://bitcoin.org/en/developer-reference#compactsize-unsigned-integers>
Parameters**i** (*int*)**Return type**

bytes

`bitcoinutils.utils.parse_compact_size(data)`

Parse variable integer. Returns (count, size)

Parameters

data (*bytes*)

Return type

tuple

`bitcoinutils.utils.get_transaction_length(data)`

Return length of a transaction, including handling for SegWit transactions.

Parameters

data (*bytes*)

Return type

int

`bitcoinutils.utils.is_address_bech32(address)`

Returns if an address (string) is bech32 or not

Parameters

address (*str*) – The address to check

Returns

True if the address is bech32, False otherwise

Return type

bool

`bitcoinutils.utils.vi_to_int(byteint)`

Converts varint bytes to int

Parameters

byteint (*bytes*)

Return type

Tuple[int, int]

`bitcoinutils.utils.add_magic_prefix(message)`

Required prefix when signing a message

Parameters

message (*str*)

Return type

bytes

`bitcoinutils.utils.tagged_hash(data, tag)`

Return a BIP-340 style tagged hash.

Tagged hashes ensure that hashes used in one context cannot be used in another. A tagged hash is `SHA256(SHA256(tag) || SHA256(tag) || data)`.

Parameters

- **data** (*bytes*)
- **tag** (*str*)

Return type

bytes

`bitcoinutils.utils.calculate_tweak(pubkey, scripts)`

Calculates the tweak to apply to the public and private key when required.

Parameters

- **pubkey** (*PublicKey*)
- **scripts** (*None* | *Script* | *list[Script]* | *list[list[Script]]* | *bytes*)

Return type

int

`bitcoinutils.utils.tapleaf_tagged_hash(script)`

Calculates the tagged hash for a tapleaf

Parameters

script (*Script*) – The script to calculate the tagged hash for

Returns

The tagged hash of the tapleaf

Return type

bytes

`bitcoinutils.utils.tapbranch_tagged_hash(thashed_a, thashed_b)`

Calculates the tagged hash for a tapbranch

Parameters

- **thashed_a** (*bytes*) – First tagged hash
- **thashed_b** (*bytes*) – Second tagged hash

Returns

The tagged hash of the tapbranch

Return type

bytes

`bitcoinutils.utils.negate_privkey(key)`

Negate private key, if necessary

Parameters

key (*bytes*)

Return type

str

`bitcoinutils.utils.tweak_taproot_pubkey(internal_pubkey, tweak)`

Tweaks the public key with the specified tweak. Required to create the taproot public key from the internal key.

Parameters

- **internal_pubkey** (*bytes*)
- **tweak** (*int*)

Return type

Tuple[bytes, bool]

`bitcoinutils.utils.tweak_taproot_privkey(privkey, tweak)`

Tweaks the private key before signing with it. Check if public key's y is even and negate the private key before tweaking if it is not.

Parameters

- **privkey** (*bytes*)

- **tweak** (*int*)

Return type

bytes

`bitcoinutils.utils.b_to_h(b)`

Converts bytes to hexadecimal string

Parameters

b (*bytes*)

Return type

str

`bitcoinutils.utils.h_to_b(h)`

Converts hexadecimal string to bytes

Parameters

h (*str*)

Return type

bytes

`bitcoinutils.utils.h_to_i(hex_str)`

Converts a string hexadecimal to a number

Parameters

hex_str (*str*)

Return type

int

`bitcoinutils.utils.i_to_h64(i)`

Converts an int to a string hexadecimal (padded to 64 hex chars)

Parameters

i (*int*)

Return type

str

`bitcoinutils.utils.b_to_i(b)`

Converts a bytes to a number

Parameters

b (*bytes*)

Return type

int

`bitcoinutils.utils.i_to_b32(i)`

Converts a integer to bytes

Parameters

i (*int*)

Return type

bytes

`bitcoinutils.utils.i_to_b(i)`

Converts a integer to bytes

Parameters

i (*int*)

Return type

bytes

2.20 Proxy API

exception `bitcoinutils.proxy.RPCException(rpc_error)`

Bases: Exception

Raised when the Bitcoin Core JSON-RPC interface returns an error.

Parameters

rpc_error (*Dict[str, Any]*)

Return type

None

code

The JSON-RPC error code (e.g. -32600 for invalid request).

Type

int | None

message

Human-readable description returned by Bitcoin Core.

Type

str | None

error

The raw error dict from the JSON-RPC response.

Type

dict

class `bitcoinutils.proxy.NodeProxy(rpcuser, rpcpassword, host=None, port=None, timeout=30, use_https=False, max_retries=3, backoff_base=0.5)`

Bases: object

JSON-RPC proxy for a Bitcoin Core node.

Supports any RPC command via dynamic attribute access:

```
proxy = NodeProxy("rpcuser", "rpcpassword")
height = proxy.getblockcount()
block_hash = proxy.getblockhash(height)
```

Parameters

- **rpcuser** (*str*) – RPC username as defined in `bitcoin.conf`.
- **rpcpassword** (*str*) – RPC password as defined in `bitcoin.conf`.
- **host** (*str*, *optional*) – Host where the Bitcoin node resides (default 127.0.0.1).

- **port** (*int*, *optional*) – Port to connect to. Uses the default port for the active network (set via `bitcoinutils.setup.setup()`).
- **timeout** (*int*, *optional*) – HTTP timeout in seconds (default 30).
- **use_https** (*bool*, *optional*) – If *True* use HTTPS; otherwise plain HTTP (default *False*).
- **max_retries** (*int*, *optional*) – Maximum number of retries on retrieable connection errors (default 3). Set to 0 to disable retries.
- **backoff_base** (*float*, *optional*) – Base delay in seconds for exponential back-off between retries (default 0.5). The *n*-th retry waits `backoff_base * 2^(n-1)`.

Raises

ValueError – If *rpcuser* or *rpcpassword* is empty / not provided.

batch_(*rpc_calls*)

Send a batch JSON-RPC request.

Parameters

rpc_calls (*list of lists*) – Each inner list is `["method", param1, param2, ...]`.

Returns

The results of each call, in the same order.

Return type

list

Raises

RPCException – If any individual call returns an error.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`